



Faculty of Sciences of Monastir
Bachelor's degree in computer science
Software Engineering

**LEXICO: Human-Oriented Programming
Language with an
AI-Assisted Compiler**

End-of-Studies Project Report

Presented by:

Gharbi Haythem

Supervised by:

Prof. Jallouli Malika

Academic Year 2025-2026

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

﴿ وَقُلْ رَبِّ زِدْنِي عِلْمًا ﴾

سورة طه - الآية 114

عَنِ النَّبِيِّ ﷺ أَنَّهُ قَالَ :

مَنْ سَلَكَ طَرِيقًا يَلْتَمِسُ فِيهِ عِلْمًا

سَهَّلَ اللَّهُ لَهُ بِهِ طَرِيقًا إِلَى الْجَنَّةِ

رواه مسلم

Tribute

This work is dedicated to the people who stood behind me, not for the result, but for the person I was becoming along the way.

To my beloved mother, source of endless love, kindness and wisdom. Your unwavering presence has been my guiding light through every challenge. Your constant encouragement and heartfelt advice laid the foundation for my success.

To my dear father, a role model of integrity and resilience. Your support has been my foundation through every challenge. In my hardest moment your patience and belief in me have shaped who I am, always stronger than my fears.

To my cherished siblings, warmth, enthusiasm and loyalty have been pillars of comfort and motivation. You've stood by me, uplifted me, and reminded me of the power of unity and family.

To my precious moonlight, tender as a sunflower, carrying warmth even across distance. Your thoughtful listening, encouragement, and gentle presence became the constant source of strength that sustained me throughout this journey.

I am forever grateful.

Acknowledgements

I would like to express my sincere gratitude to my academic supervisor, Mrs. Jallouli Malika, guidance, supervision, and continuous support throughout this project. Her expertise and valuable contribution were essential to its completion.

I am also grateful to the Faculty of Sciences of Monastir for providing a supportive academic environment and quality education throughout my studies. The knowledge and skills acquired there played a key role in the realization of this work.

My sincere appreciation goes to the members of the jury for their time and effort in evaluating this project. Their insightful remarks and constructive feedback significantly helped improve its quality, and I highly value their professionalism and dedication.

Finally, I extend my heartfelt thanks to my family and to a special person for their continuous encouragement, moral support, and motivation, which were essential throughout this journey.

Haythem Gharbi

Abstract

This work presents LEXICO, a human-oriented programming language coupled with an AI-assisted compiler architecture. The compiler backend follows a deterministic design implemented in C++, integrating Flex-based lexical analysis, Bison GLR parsing, semantic validation, and LLVM C API-driven IR generation with JIT execution.

To improve developer productivity, an AI diagnostics layer is introduced at the orchestration level to interpret compilation failures and provide corrective guidance without replacing core compilation stages.

To validate the language in practice, a functional password manager CLI application was developed entirely in LEXICO, demonstrating key runtime capabilities such as authentication, file persistence, and modular program structure.

The resulting system combines human-friendly language design, reliable compiler architecture, and AI-enhanced usability for practical software development.

Keywords: LEXICO, human-oriented programming language, AI-assisted compiler, deterministic compilation, Flex, Bison GLR, semantic analysis, LLVM C API, IR generation, JIT execution, developer productivity, compiler diagnostics.

List of Content

List of Figures

List of Tables

List of Acronyms

General Introduction

1 Project Context and State of the Art

1.1	Introduction.....	2
1.2	Project Background.....	2
1.2.1	Domain context.....	2
1.2.2	Motivation for the project.....	2
1.3	Problem Statement.....	2
1.4	Related Work.....	4

2 Conceptual and Architectural Design

2.1	Introduction.....	5
2.2	Conceptual Foundations.....	5
2.2.1	Human-Oriented Language Principles.....	5
2.2.2	Intuitive Syntax, Semantics, and Usability Goals.....	6
2.3	System Overview	
2.3.1	High-Level Architecture Diagram.....	9
2.3.2	Main Components.....	10

2.3.3	Data Flow and Module Interactions.....	13
2.4	Compiler Architecture	
2.4.1	Lexical Analysis.....	16
2.4.2	Abstract Syntax Tree.....	18
2.4.2	Syntax Analysis.....	19
2.4.4	Semantic Analysis and Symbol Table.....	20
2.4.5	Code Generation.....	22
2.4.6	Error Handling and Reporting.....	24
2.4.7	Compilation Driver and Workflow.....	24
2.5	AI-Assisted Diagnostics Design	
2.5.1	Motivation for AI Integration.....	27
2.5.2	Architecture of the Diagnostics Layer.....	27
2.5.3	Error Detection and Suggestion Mechanism.....	28
2.5.4	User Interaction and Feedback Loop.....	28
2.6	Design Rationale and Choices	
2.6.1	Language Design Decisions.....	29
2.6.2	Compiler Technology Choices.....	30
2.6.3	AI Model and Tool Selection.....	30
2.6.4	Trade-offs and Limitations.....	30
3	Implementation	
3.1	Introduction.....	31
3.2	Overview of Implementation Approach	
3.2.1	Technology Stack.....	31

3.2.2	Development Workflow.....	33
3.2.3	Key Implementation Challenges.....	33
3.3	Lexical Analyzer Implementation	
3.3.1	Tool Selection.....	33
3.3.2	Token Definitions.....	34
3.3.3	Handling Comments and whitespace.....	34
3.3.4	Custom Lexical Rules.....	35
3.3.5	Testing the Lexer.....	35
3.4	Syntax Analyzer Implementation	
3.4.1	Tool Selection.....	35
3.4.2	Grammar Design.....	36
3.4.3	Ambiguity Resolution.....	36
3.4.4	Parser Actions and Error Recovery.....	36
3.4.5	Testing the Parser	37
3.5	AST Construction	
3.5.1	AST Node Types and Hierarchy.....	37
3.5.2	AST Constructions During Parsing.....	39
3.5.3	AST Traversal and Manipulation.....	39
3.6	Semantic Analysis and Symbol Table	
3.6.1	Symbol Table Structure.....	39
3.6.2	Scope Management.....	40
3.6.3	Type Checking Implementation.....	41
3.7	Code Generation	

3.7.1	Target Code Format.....	41
3.7.2	Code Emission Strategies.....	41
3.7.3	Runtime Helper Integration.....	42
3.7.4	Output File Handling.....	43

4 Validation and Evaluation

4.1	Introduction.....	47
4.2	Validation Approach.....	47
4.3	Password Manager Case Study	
4.3.1	Overview and Goals.....	48
4.3.2	Features Implemented.....	48
4.3.3	Compilation and Execution Results.....	49
4.4	Compiler Validation Scenarios	
4.4.1	Lexical and Syntax Error Detection.....	49
4.4.2	Semantic Error Detection.....	50
4.4.3	AI-Assisted Diagnostics Evaluation.....	52
4.5	Summary of Results.....	53

General Conclusion

References

Annex A: Grammar & Parser

Annex B: Lexer & Tokenization

Annex C: AST Structures

Annex D: Semantic Analysis

Annex E: Code Generation & JIT

Annex F: Tests & Limitations

List of Figures

1.1	Waterfall Methodology.....	3
2.1	LEXICO – human oriented language principles.....	6
2.2	Semantic Consistency in LEXICO Design.....	7
2.3	LEXICO Usability Design Overview.....	8
2.4	LEXICO General Pipeline Architecture.....	9
2.5	LEXICO General Architecture Organization layers.....	13
2.6	LEXICO Data Flow and Module Interactions.....	14
2.7	Lexical Analysis in LEXICO.....	16
2.8	LEXICO Lexer INDENT/DEDENT.....	16
2.9	Lexical Analysis in LEXICO	17
2.10	LEXICO Token Categories.....	17
2.11	Token Examples From actual rules in lexico.l.....	17
2.12	LEXICO AST Nodes Families.....	18
2.13	LEXICO AST Nodes Categories.....	19
2.14	Bison GLR Parsing Flow — Token Stream to AST.....	20
2.15	Symbol Table Scope Stack and Variable Lifecycle.....	20
2.16	Two-Phase Semantic Analysis — Signature Collection Then Validation.....	21
2.17	Code Generation Pipeline — AST Node to LLVM IR Instruction.....	22
2.18	LLVM IR Example for a Simple LEXICO Assignment.....	23
2.19	Error Reporting Flow — Stage Diagnostics to Driver.....	24

2.20	Compiler Workflow Architecture.....	26
2.21	AI Diagnostics Layer Architecture — Driver to LLM and Back.....	28
2.22	User Feedback Loop — Standard Error + AI Suggestion Output.....	29
3.1	LEXICO Technology Stack Overview	32
3.2	INDENT/DEDENT Stack Mechanism in lexico.l.....	35
3.3	Language overview Categories, roles, examples, and key tokens.....	36
3.4	AST Node Type Hierarchy	38
3.5	Symbol Table Structure and Entry Layout.....	40
3.6	LLVM IR Emission for an Assignment Statement.....	42
4.1	Password Manager CLI — Application Architecture in LEXICO.....	48
4.2	Password Manager Source Code Excerpt — File I/O and Routine Call.....	48
4.3	Password Manager Use Case.....	49
4.4	Syntax Error Detection — Compiler Diagnostic Output Example.....	50
4.5	Semantic Error Detection — Undeclared Variable and Type Mismatch Examples..	51
4.6	Compiler Output for a Missing Semicolon.....	52
4.7	AI Suggestion Output for a Missing Semicolon.....	52

List of Tables

1.1	Comparison of related Work and LEXICO.....	4
2.1	LEXICO adaptation for Human-Oriented style.....	7
2.2	Entry point layer Components of the LEXICO Architecture.....	10
2.3	Front-end layer Components of the LEXICO Architecture.....	11
2.4	Back-end layer Components of the LEXICO Architecture.....	11
2.5	Output layer Components of the LEXICO Architecture.....	12
2.6	Optional layer Components of the LEXICO Architecture.....	12
2.7	Module Inputs and Outputs in the LEXICO Pipeline.....	15
3.1	LEXICO Technology Stack.....	32
3.2	LEXICO Token Categories and Representative Examples.....	34
4.1	Validation Summary — Test Coverage by Stage and Error Category.....	53

List of Acronyms

AST	Abstract Syntax Tree
LR	Left to Right
GLR	Generalized LR
LALR	Look-Ahead LR
IR	Intermediate Representation
JIT	Just-In-Time
MCJIT	Machine Code JIT
SSA	Static Single Assignment
NFA	Nondeterministic Finite Automaton
ABI	Application Binary Interface
LLVM	Low Level Virtual Machine
API	Application Programming Interface
CLI	Command-Line Interface
JSON	JavaScript Object Notation
HTTPS	Hypertext Transfer Protocol Secure
HTTP	Hypertext Transfer Protocol
LLM	Large Language Model
OOP	Object-Oriented Programming
DSL	Domain Specific Language
BOL	Beginning-Of-Line
DQ	Dedent Queue

General Introduction

General Introduction

The rapid growth of digital technologies has changed how software is developed, but programming remains challenging due to strict syntax and error-prone code. This gap between human thinking and machine requirements makes it hard to express ideas clearly in code. This project introduces LEXCIO as a human-oriented programming language with an AI-assisted compiler designed to bridge this gap by making programming more intuitive while preserving reliable and well-defined compilation.

Developers, students, and non-specialists often spend significant time fixing syntax-related issues instead of focusing on logic and problem-solving [1], and that compiler error messages are often difficult for learners to interpret and act upon effectively [2]. To address this problem, programming tools should be designed to reduce unnecessary syntactic friction, allowing users to focus on algorithmic reasoning, problem decomposition, and solution quality rather than low-level writing constraints.

LEXICO overcomes these challenges and translates this goal into a practical system focusing on simplifying how programs are written by reducing syntactic complexity while preserving reliable and well-defined compilation behavior. It is designed to provide a more natural way of expressing instructions, especially enabling beginners users to interact with programming concepts in a clearer and more intuitive manner.

LEXICO strengthens practical usability by integrating an AI-assisted diagnostics layer from error detection to correction through clearer guidance during development. This improves debugging efficiency and reduces trial-and-error cycles while maintaining a consistent and predictable compiler workflow. To evaluate the system in a realistic context, a complete password manager CLI applications was implemented fully using LEXICO, including credential handling, persistent storage, and modular organization. This demonstrates that the project provides not only conceptual improvements but also practical utility in real development scenarios.

This report is organized into four chapters. The first chapter introduces the project context, defines the core problem, and reviews related work to position the proposed contribution. The second chapter presents the conceptual and architectural design of LEXICO and its compiler, highlighting the main design choices and system structure. The third chapter details the implementation process, including the compiler workflow and the integration of AI-assisted diagnostics within the compilation cycle. Finally, the fourth chapter evaluates the proposed approach through practical validation scenarios, including a password manager case study, and concludes with the main outcomes, limitations, and future improvement directions.

Chapter 1 :

Project Context and State of the Art

1

Project Context and state of the Art

1.1 Introduction

This chapter sets the context for LEXICO by summarizing the state of programming language design, user challenges, and the motivation for human-oriented tools. It also defines the core problem and reviews related work to position LEXICO within the broader research landscape.

1.2 Project Background

This section summarizes the project's context, key trends and challenges, and motivations for creating LEXICO.

1.2.1 Domain Context

The project focuses on programming language design and compiler technology to improve accessibility. While general-purpose languages dominate, DSLs and educational tools are growing to make programming easier, and AI is increasingly used for smarter code assistance. LEXICO follows this direction by combining human-oriented syntax with AI-assisted compilation.

1.2.2 Motivation for the Project

Many developers struggle with strict syntax and unclear errors, which slows learning and productivity. Current tools don't fully provide a natural, supportive experience. This project is motivated by the need for a language and compiler that reduce syntactic barriers, give clearer guidance, and make programming more accessible.

1.3 Problem Statement

Programming languages often require strict syntax and provide error messages that are difficult for many users to interpret, creating a gap between human reasoning and machine requirements. This leads to slow learning, frequent mistakes, and inefficient development. LEXICO addresses

this by offering a human-oriented language and an AI-assisted compiler that reduce syntactic friction and provide clearer guidance throughout the compilation process.

To develop and validate this solution, the project followed the phases below:

- **Analysis and Problem Identification:** study user difficulties and limitations in existing tools.
- **Requirements Definition:** specify language goals, compiler features, and usability targets.
- **Design Phase:** define the language syntax/semantics and compiler architecture.
- **Implementation Phase:** build the compiler pipeline and integrate AI diagnostics.
- **Validation & Evaluation:** test the system and validate it using the password manager case study.

This structured sequence reflects a Waterfall methodology.

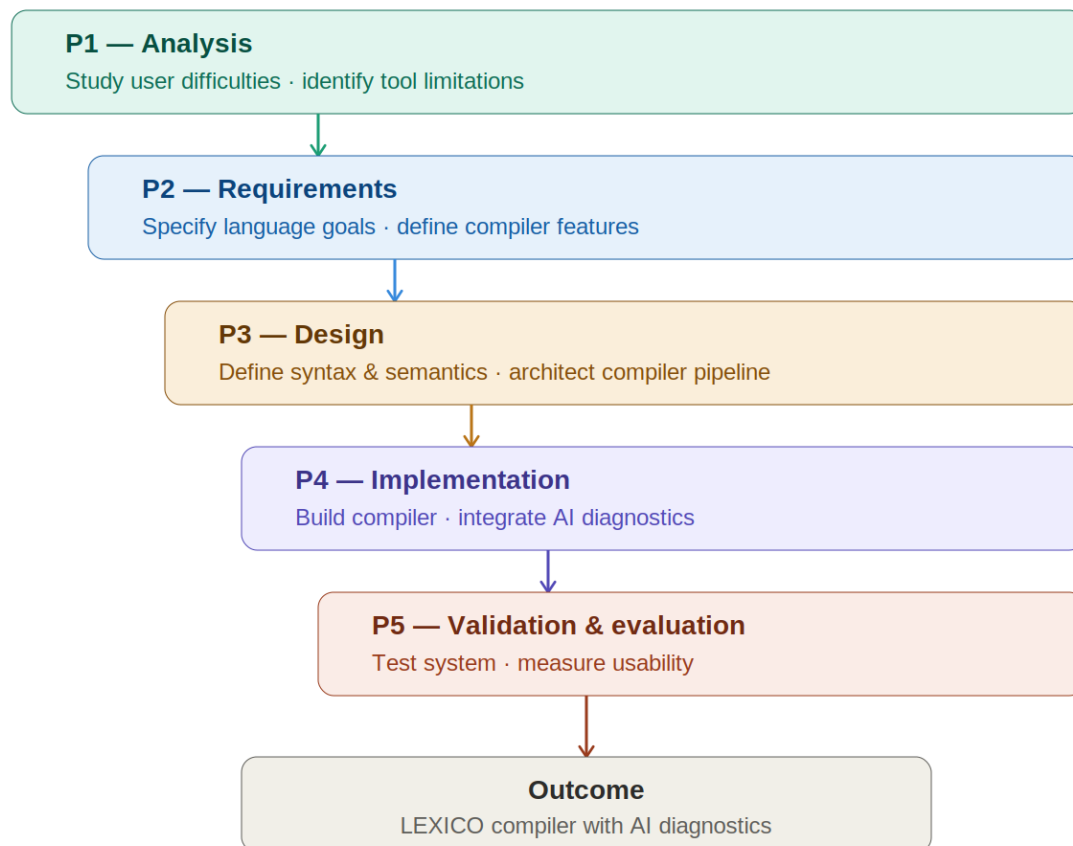


Figure 1.1: Waterfall Methodology

1.4 Related Work

Educational tools like Scratch and Blockly reduce syntax errors through visual blocks. Readable languages such as Python help beginners but still enforce strict syntax and often generate confusing errors. Studies in computing education show that syntax mistakes and unclear compiler feedback are major barriers for novices. AI-assisted tools improve productivity but still rely on traditional languages.

LEXICO stands out by combining a human-oriented language with an AI-assisted compiler, reducing syntactic friction while also improving error guidance within a well-defined compilation pipeline.

Table 1.1: Comparison of related Work and LEXICO

Approach Example	Core Idea	Strengths	Limitations	How LEXICO Differs
Scratch / Blockly	Visual blocks avoid syntax	Easy for beginners	Limited expressiveness	LEXICO keeps textual coding but simplifies syntax
Python	Readable syntax	Community supported	Still strict syntax and cryptic errors.	LEXICO reduces syntactic friction and clearer diagnostics
Education studies	Identify common novice errors	Evidence-based	Doesn't provide a tool	LEXICO implements a practical solution
AI code assistants	Suggestions and diagnostics	Productivity boost	Built on traditional languages	LEXICO integrates AI directly into compiler

Chapter 2 :

Conceptual and Architectural Design

2

Conceptual and Architectural Design

2.1 Introduction

This chapter summarizes LEXICO's design, architecture, compiler pipeline, AI-assisted diagnostics, guiding principles, key choices, rationale, and trade-offs.

2.2 Conceptual Foundations

This section outlines LEXICO's core ideas: human-centered programming, intuitive syntax, and accessibility goals.

2.2.1 Human-Oriented Language Principles

Human-oriented languages favor natural expression over machine-centric rules. LEXICO prioritizes clarity, readability, and reduced syntax so users focus on logic, not low-level details.

LEXICO adopts several core principles to achieve this:

- **Natural expression of intent:**
The language encourages writing that resembles natural reasoning, using keywords and structures that are easy to interpret.
- **Reduced syntactic friction:**
LEXICO minimizes unnecessary punctuation and rigid formatting rules, which are common sources of mistakes for beginners.
- **Readability over brevity:**
Instead of prioritizing compact code, LEXICO emphasizes readability and explicitness.

- **Progressive learning:**

The language supports gradual learning: simple programs can be written with minimal syntax, while more advanced features can be adopted as the user gains experience.

- **Error-tolerant design:**

The compiler and language design are aligned to detect common mistakes and guide the user with meaningful feedback, reducing frustration and improving learning outcomes.

These principles guide all subsequent design choices in LEXICO, from its syntax and semantics to the compiler's diagnostic behavior.

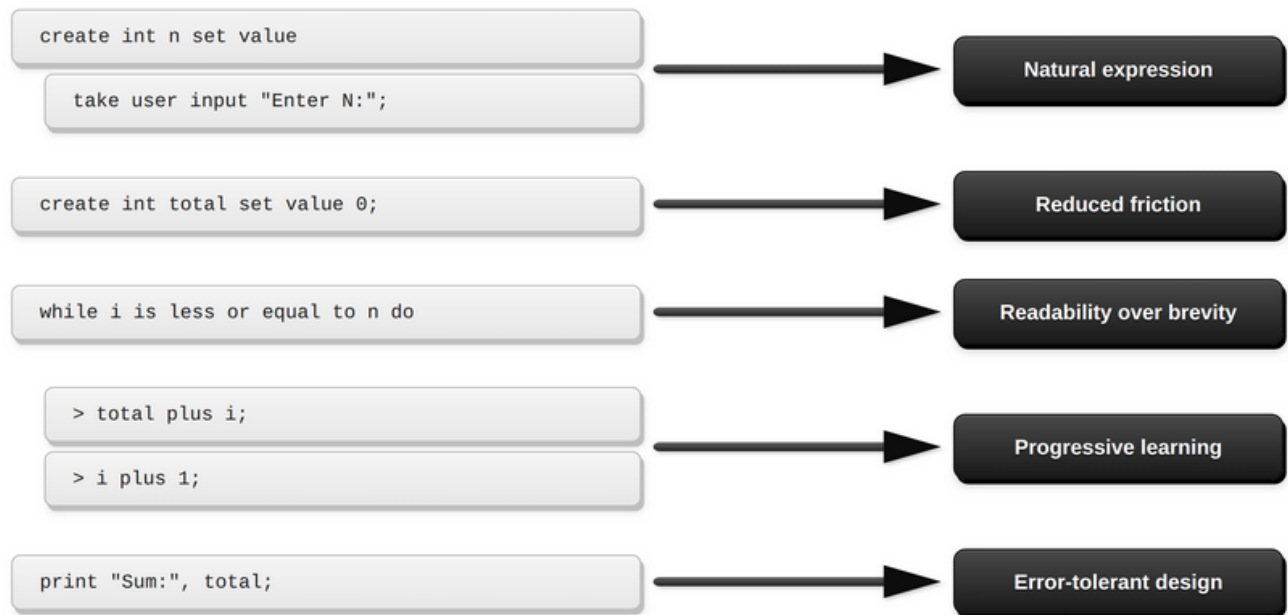


Figure 2.1: LEXICO — human oriented language principles

2.2.1 Intuitive Syntax, Semantics, and Usability Goals

LEXICO's syntax is designed to be readable and close to natural reasoning, reducing the need for symbols and rigid punctuation. Statements follow clear, word-based structures that make intent explicit and reduce common syntax mistakes as it is presented in Figure 2.2.

This also lowers the barrier for new users who may struggle with symbolic operators or strict formatting rules in traditional languages.

Table 2.1: LEXICO adaptation for Human-Oriented style

Line	Instruction	Description
1	create int x set value 5;	Declare an integer variable x and initialize it to 5.
2	set x to x plus 1;	Increment x by 1 ($x = x + 1$, so x becomes 6).
3	print x;	Output the current value of x (prints 6).

Semantically, the language stays predictable: each statement has one clear meaning, and control flow uses simple readable constructs.

its clarity in evaluation order type behavior and updates helps learners build a reliable mental model.

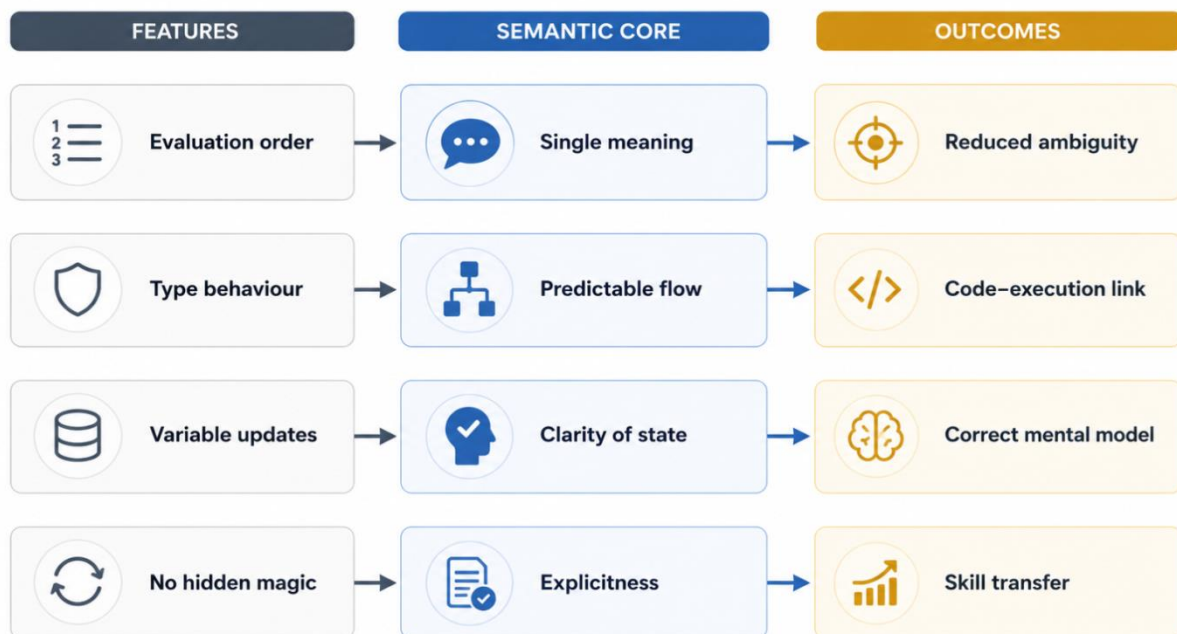


Figure 2.2: Semantic Consistency in LEXICO Design

As presented in Figure 2.3 from a usability perspective LEXICO prioritizes clarity over brevity, supporting gradual learning and reducing cognitive load for beginners. It encourages self-documenting code and pairs with compiler diagnostics, helping errors stay readable while making programs easier to write, understand and debug.

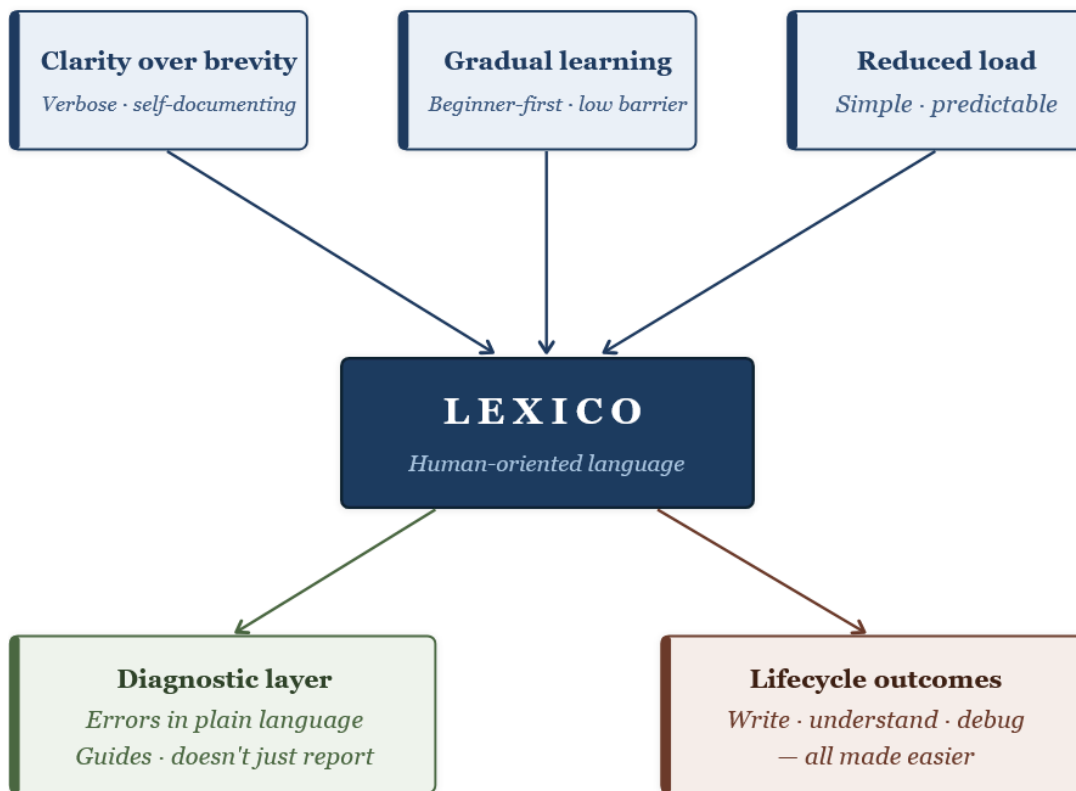


Figure 2.3: LEXICO Usability Design Overview

2.3 System Overview

This section outlines the LEXICO system space structure and how its components interact from source to compiled output. It covers preprocessing lexical analysis parsing AST construction semantic validation and LLVM code generation, with the driver coordinating load expansion, error handling, and optional AI-assisted diagnostics. The backend can emit LLVM IR or execute it directly via MCJIT for transparency and practical usability

2.3.1 High-Level Architecture Diagram

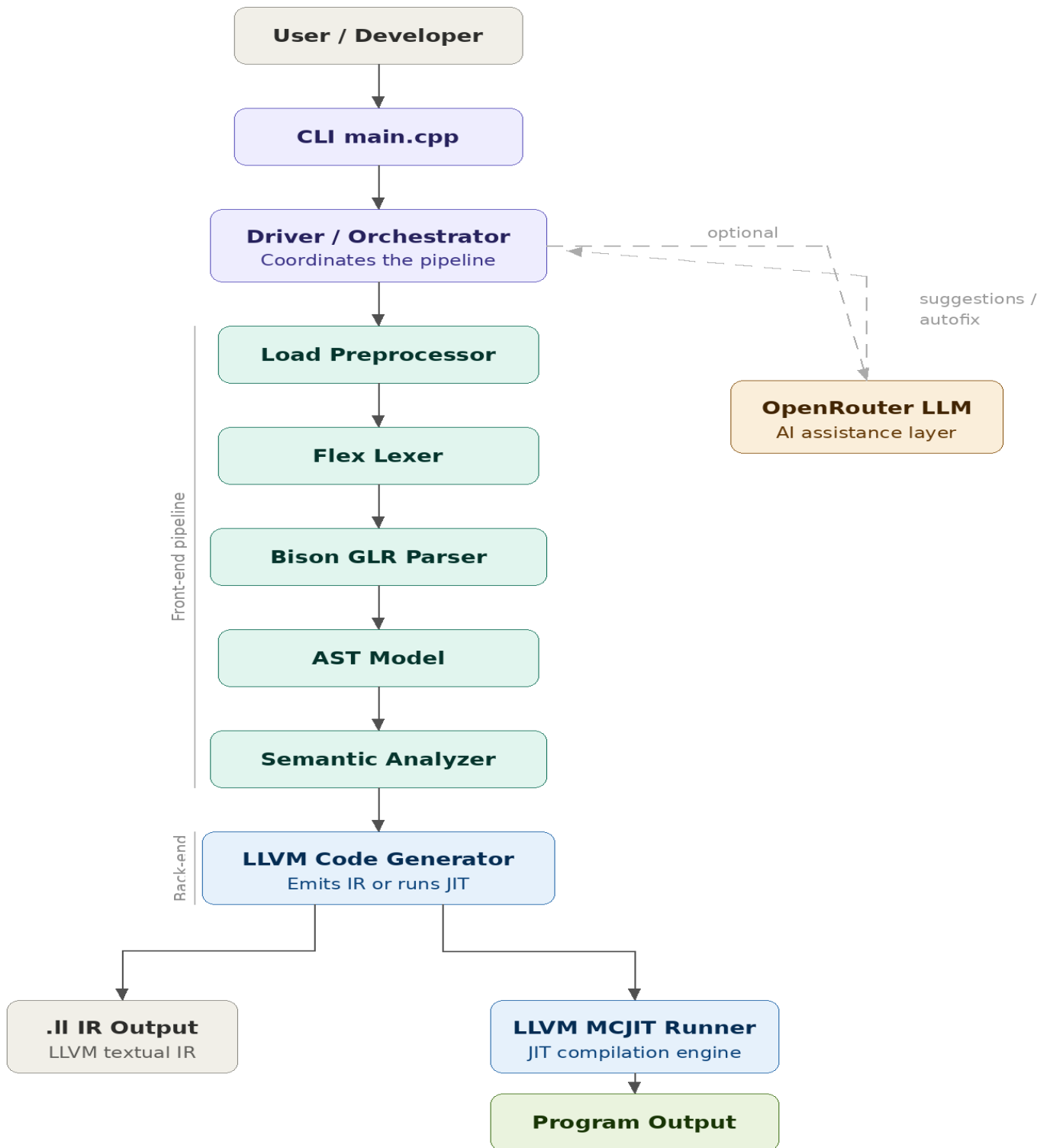


Figure 2.4: LEXICO General Pipeline Architecture

LEXICO is implemented as a pipeline based compiler where each stage produces a defined artifact before passing control to next.

- **main.cpp** validates input and dispatches to the driver, which expands loads and orchestrates the pipeline.
- **lexico.l** tokenizes keywords and indentation tokens, feeding the GLR parser, which builds the AST through constructors in ast.hpp and ast.cpp.
- **syntab.cpp** enforces declaration, type, signature and context constraint validation rules.
- **codegen.cpp** generates LLVM IR, registers runtime helper bindings, and executes the module with MCJIT directly.

An optional AI layer in the driver activates on parse, semantic, and codegen failures to offer suggestions or autofix, preserving deterministic compilation while improving usability and error recovery.

2.3.2 Main Components

As the Figure2.5 presents the architecture is organized into five layers, each with a clear role.

- **The Entry-point** is user-facing surface of the system, CLI validates file paths flags and output mode enforces the.lx extension and forwards execution to the driver so it coordinates pipeline stages manages shared state and diagnostics and invokes the optional AI layer to suggest or autofix errors.

Table 2.2 : Entry point layer Components of the LEXICO Architecture

Component	Role and Responsibility
User / Developer	Writes the .lx code, runs the compiler, and reads the output or errors.
CLI main.cpp	Parses command-line arguments and delegates control to the Driver/Orchestrator.
Driver / Orchestrator	Runs compilation stages in order, manages shared state and errors, and invokes the optional LLM for auto-fix suggestions when needed.

- **Front-end layer** turns raw source into a validated structure: load preprocessing expands load directives, the lexer tokenizes keywords and literals, the GLR parser builds the AST, and semantic analysis verifies symbols, types, signatures, and context rules. By the end, the program is structurally and semantically consistent.

Table 2.3 : Front-end layer Components of the LEXICO Architecture

Component	Role and Responsibility
Load Preprocessor	Handles source-level transformations before lexing.
Flex Lexer	Scans the preprocessed character stream using regular-expression rules and emits a sequence of typed tokens consumed by the parser.
Bison GLR Parser	Applies a generalized LR grammar to the token stream, resolving structural ambiguities at parse time, and constructs a concrete syntax tree passed to the AST builder.
AST Model	Models program syntax as a typed, traversable AST shared across all later analysis and transformation passes.
Semantic Analyzer	Enforces AST language semantics annotates nodes with type information and emits diagnostics for semantic errors.

- **Back-end layer** converts validated AST into LLVM IR, maps LEXICO constructs to LLVM instructions and runtime helpers, runs verification plus basic optimizations, and outputs portable IR for execution or export.

Table 2.4 : Back-end layer Components of the LEXICO Architecture

Component	Role and Responsibility
LLVM Code Generator	Turns annotated AST into LLVM IR, applying target independent optimizations as the bridge between LEXICO's front-end logic and LLVM's backend machinery.

- **Output layer** either writes a .ll IR file for inspection or runs it directly via LLVM MCJIT to produce output. The optional AI layer, attached to the driver, can provide clearer diagnostics or auto-fix suggestions on failure without changing the deterministic pipeline.

Table 2.4: Output layer Components of the LEXICO Architecture

Component	Role and Responsibility
.ll IR Output	Serializes the generated LLVM IR to a human-readable .ll textfile.
LLVM MCJIT Runner	Compiles LLVM IR to native machine code at runtime using MCJIT .
Program Output	The observable result of executing the compiled program.

- **Optional layer** is the AI-assisted diagnostics and auto-fix layer handles driver failures, producing clearer guidance or suggested fixes while leaving the deterministic compiler pipeline unchanged.

Table 2.5: Optional layer Components of the LEXICO Architecture

Component	Role and Responsibility
OpenRouter LLM	AI assistance layer invoked by the Driver when compilation errors occur.

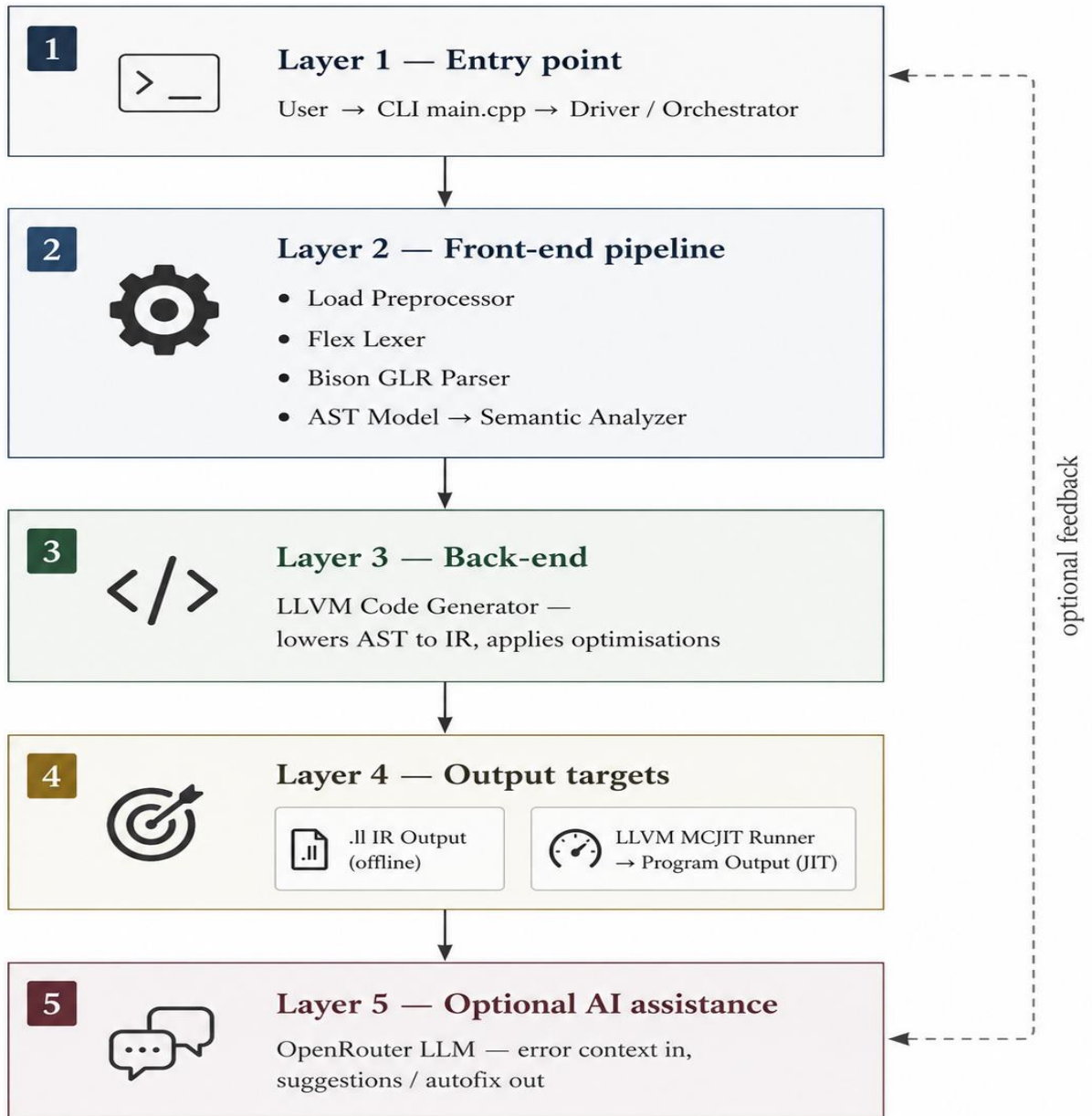


Figure 2.5: LEXICO General Architecture Organization layers

2.3.3 Data Flow and Module Interactions

During development, the LEXICO compilation flow follows a strict artifact-based pipeline to keep each stage testable and deterministic as presented in the Figure 2.6 below.

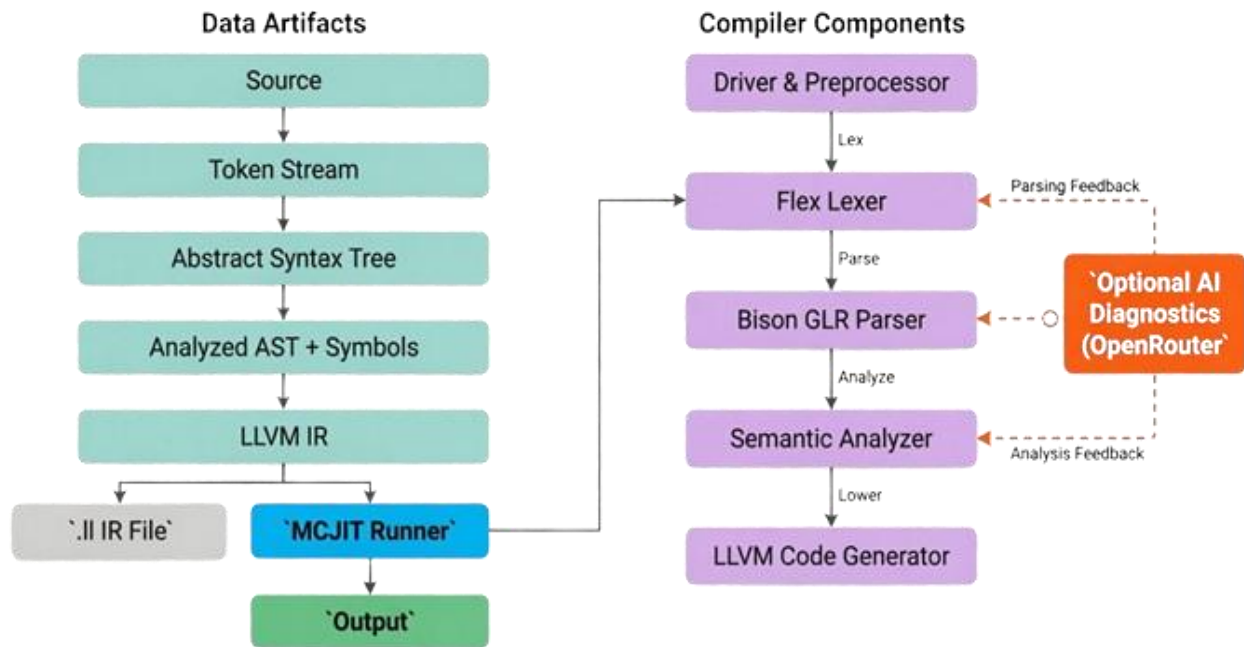


Figure 2.6: LEXICO Data Flow and Module Interactions

- **The Driver** expands loads, resolves paths, normalizes source into one buffer.
- **The Flex Lexer** lexico.l, which emits tokens with location and indentation metadata.
- **The Bison GLR** parser lexico.y consumes them to build the AST using constructors in ast.hpp and ast.cpp.
- **The Semantic Analyzer** symtab.cpp registers symbols checks duplicates, gathers signatures, infers types, validates context, and produces semantically checked AST and symbol table state.
- **The LLVM Code Generator** codegen.cpp lowers AST to LLVM IR, declares runtime helpers for collections, strings, matrices, and file I/O, verifies the module, and can emit .ll or run via LLVM MCJIT.
- **Diagnostics** route through the driver, and optional AI assistance runs only on failures, giving clear fixes without changing the deterministic pipeline.

Table 2.7: Module Inputs and Outputs in the LEXICO Pipeline

Module	Input Artifact	Output Artifact	Role
Driver / Preprocessor	.lx source files	Expanded source buffer	Expands load directives, resolves paths, and normalises input into a single buffer.
Flex Lexer	Source buffer	Token stream	Scans the source buffer and emits typed tokens.
Bison GLR Parser	Token stream	AST (Program)	Applies GLR grammar to the token stream and constructs the AST.
Semantic Analyser	AST	Validated AST + symbol / type info	Performs symbol registration, duplicate detection, type inference, and context-sensitive validation.
Code Generator	Validated AST	LLVM IR (.ll)	Lowers the validated AST to LLVM IR, declares runtime helpers and verifies the IR module
MCJIT Runner	LLVM IR	Program output	JIT-compiles the IR to native machine code and executes it in-process.

2.4 Compiler Architecture

This section traces the compiler pipeline, pipeline from raw source to execution showing how lexing parsing AST building semantic checks and codegen produce precise artifacts for the next stage. It also covers stage-specific errors driver orchestration and optional AI diagnostics

2.4.1 Lexical Analysis

As shown in the figure 2.7 lexical analysis in LEXICO lexico.l scans raw source and converts it to tokens, recognizing keywords, identifiers, literals, operators, punctuation, and layout markers while tracking line/column positions for precise diagnostics.

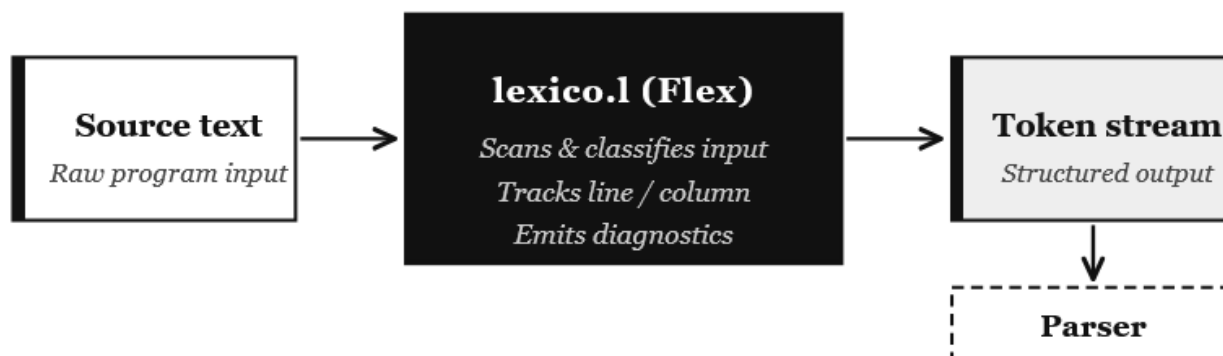


Figure 2.7: Lexical Analysis in LEXICO

Furthermore the Figure 2.8 presents LEXICO's lexer is layout-sensitive. Lines beginning with ">" markers become INDENT/DEDENT tokens via a stack, letting the parser handle nested blocks without end keywords. It also folds multi-word operators like is equal to, is greater than, and divided by into single tokens such as CMP_EQ, CMP_GT, and DIV for parsing.

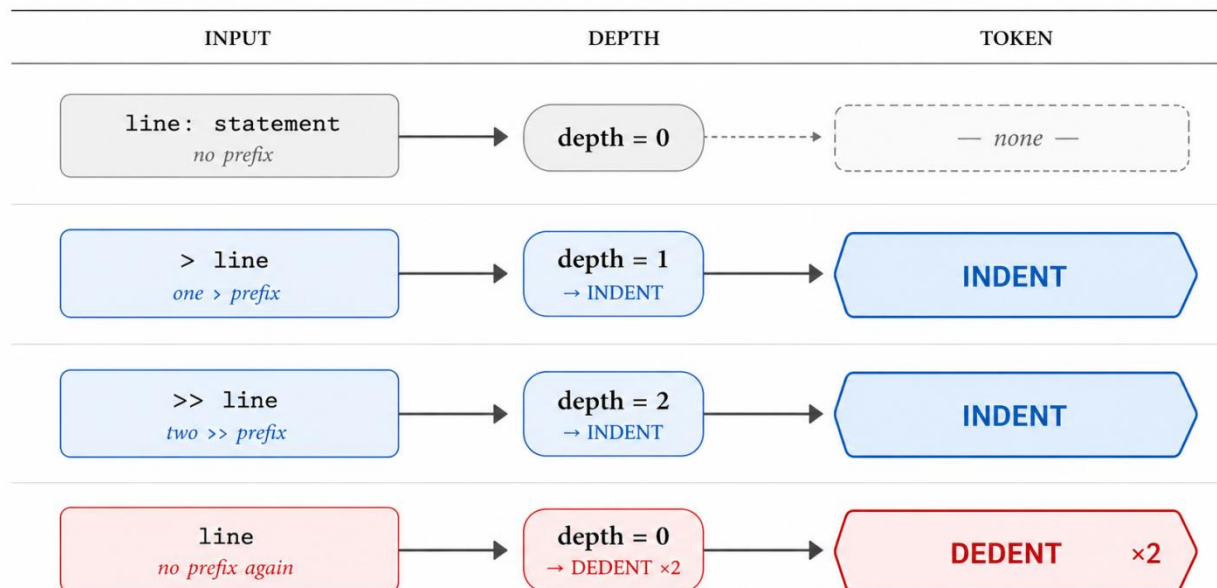


Figure 2.8: LEXICO Lexer INDENT/DEDENT

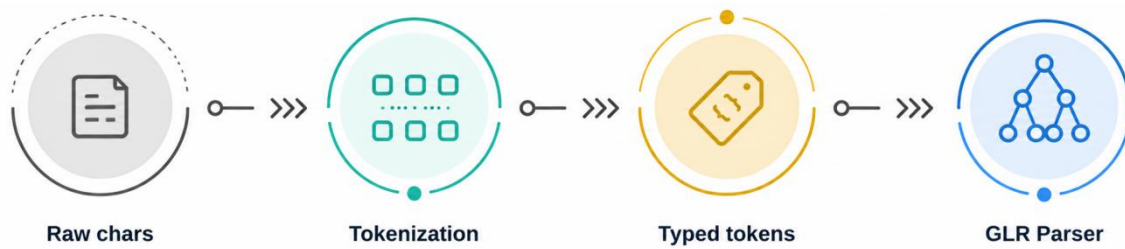


Figure 2.9: Lexical Analysis in LEXICO

Supported literal classes include integers, floats, chars, strings, and booleans. Identifiers follow the standard pattern `[a-zA-Z_][a-zA-Z0-9_]*`, while punctuation like commas, semicolons, and parentheses are emitted as their literal tokens.

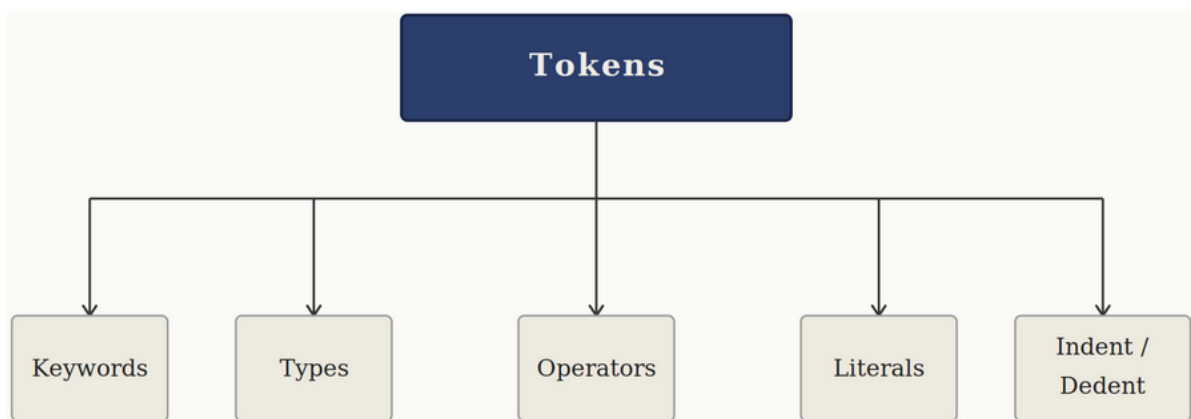


Figure 2.10: LEXICO Token Categories

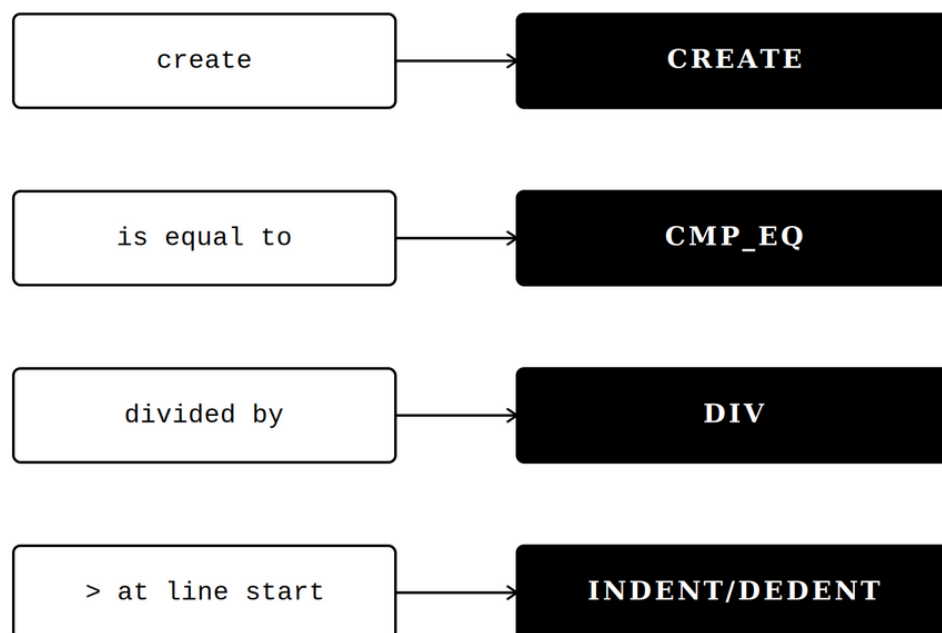


Figure 2.11: Token Examples From actual rules in `lexico.l`

2.4.2 Abstract Syntax Tree

In LEXICO, the AST is the central in-memory representation of a program. It is created by the parser `lexico.y` and defined in `ast.hpp/ast.cpp`. The AST captures the meaning of the program rather than its surface syntax, so punctuation and formatting are removed while structural intent is preserved.

The AST is composed of a small set of core node families:

- **Program:** root node that owns a list of statements.
- **Stmt:** represents declarations, assignments, control flow, I/O, routines, and file/collection operations.
- **Expr:** represents literals, identifiers, arithmetic expressions, function calls, indexing, and conversions.
- **CondExpr:** represents boolean conditions used in if, while, and other control flow.

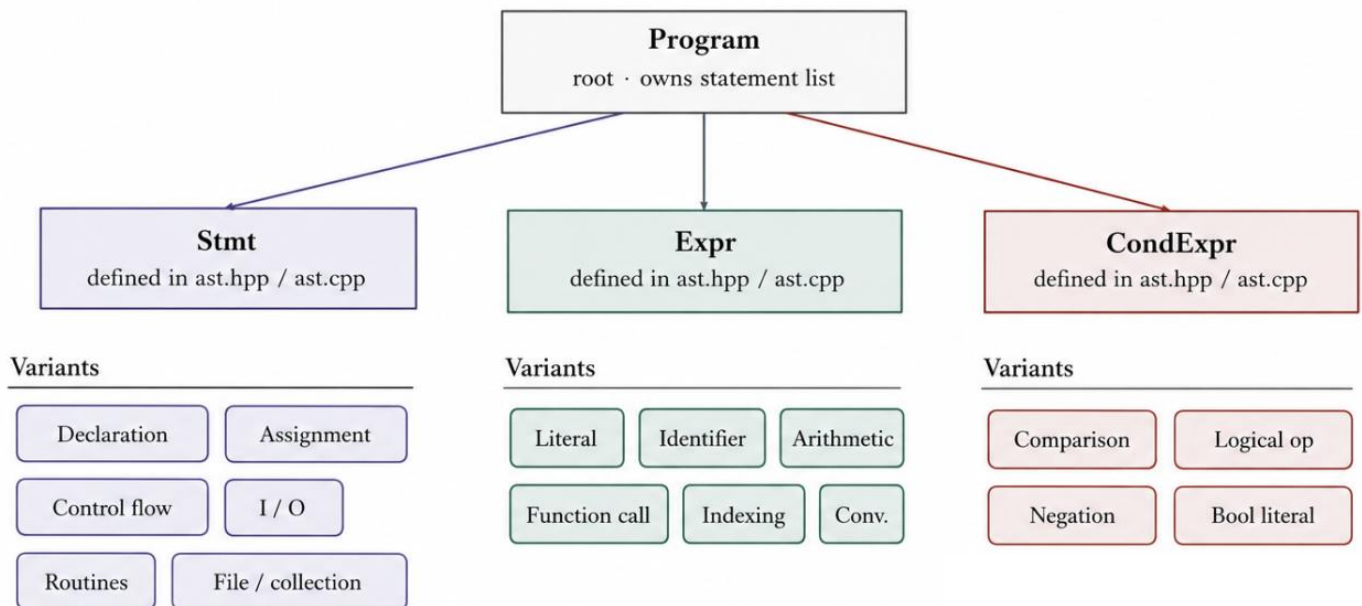


Figure 2.12: LEXICO AST Nodes Families

Each node carries a StmtKind or ExprKind tag so later stages can traverse the AST deterministically and apply the correct rules with simple switch logic. LEXICO also uses explicit ownership: every AST node owns its heap-allocated fields, and `ast_free_program()` recursively frees the tree to keep cleanup deterministic and safe.

Node Kind Category	Examples (Kinds)	Traversal Pattern	Used For
 StmtKind Statement nodes	• STMT_DECL • STMT_IF • STMT_WHILE • ...	switch(node->kind) → apply rule for this kind	 Semantic analysis
 ExprKind Expression nodes	• EXPR_INT_LIT • EXPR_BINOP • EXPR_CALL • ...	switch(node->kind) → apply rule for this kind	 Type checking
 CondExprKind Condition expr. nodes	• COND_AND • COND_OR • COND_NOT • ...	switch(node->kind) → apply rule for this kind	 Code generation

Owner	What It Owns	What It Means	How to Free
Stmt node	 Heap-allocated fields	• Owns its children/fields on the heap	 Freed recursively
Expr node	 Heap-allocated fields	• Owns its children/fields on the heap	 Freed recursively
CondExpr node	 Heap-allocated fields	• Owns its children/fields on the heap	 Freed recursively
ast_free_program()	 Entire AST (all nodes)	• Frees the whole tree safely	 Call once to free everything

Figure 2.13: LEXICO AST Nodes Categories

2.4.3 Syntax Analysis

As shown in the Figure 2.14 the syntax analysis stage in LEXICO uses Bison in GLR mode to consume the token stream from the Flex lexer and build the AST from the rules in `lexico.y`. GLR was chosen over standard LALR because LEXICO's natural-language-like syntax creates structural ambiguities that a classical LALR parser cannot resolve deterministically. When ambiguities arise, GLR keeps multiple parse states in parallel and resolves them as more tokens are read, which supports multi-word operators, indentation-based blocks, and flexible statement forms without forcing the grammar to be artificially restricted.

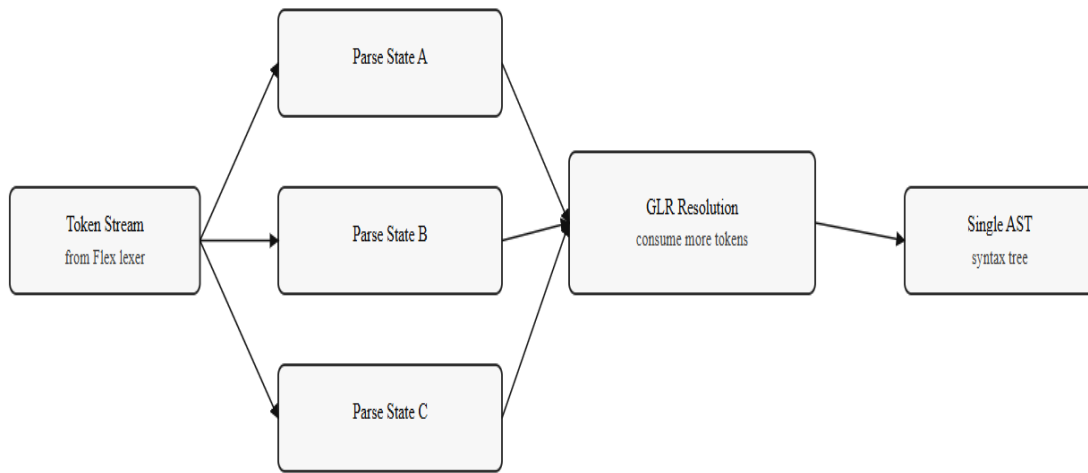


Figure 2.14: Bison GLR Parsing Flow — Token Stream to AST

2.4.4 Semantic Analysis and Symbol Table

Semantic analysis in LEXICO is performed by the module implemented in `syntab.cpp` and `syntab.hpp`. It operates on the AST produced by the parser and enforces the language's semantic rules before code generation begins. The analyzer traverses the tree in two phases: it first collects routine signatures, then validates all statements, expressions, and function calls against the accumulated symbol and type information.

The symbol table is structured as a scoped hash table. Each entry records the variable's name, declared type, kind, and scope level. A scope stack tracks nested contexts such as function bodies, loop bodies, and file blocks. When a block is entered, a new scope frame is pushed; when exited, the frame is popped and its symbols are released.

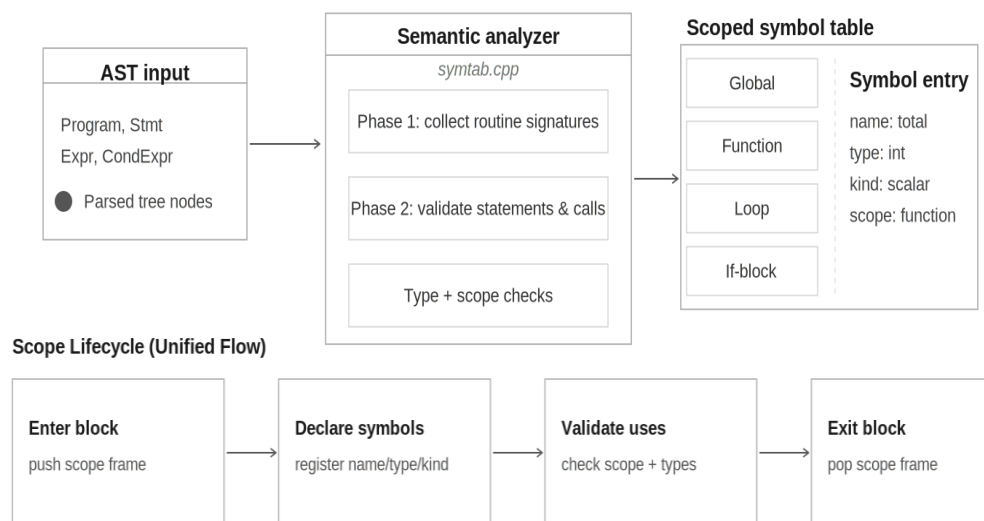


Figure 2.15: Symbol Table Scope Stack and Variable Lifecycle

The key validation tasks performed during this stage include the following. Declaration checking ensures that every variable used in an expression was previously declared with a create statement and is accessible in the current scope. Duplicate detection prevents the same identifier from being declared twice in the same scope. Type inference derives the result type of binary expressions using a compatibility matrix. Context-specific to LEXICO constructs, such as the restriction that write and read statements may only appear inside an open file block, and that set value inside a for-in loop applies to the collection slot bound to the loop variable.

Function signature collection is performed in the first traversal pass: all routine declarations are registered before statement bodies are checked, enabling mutual recursion and forward references. During the second pass, each call site is validated against the registered signature.

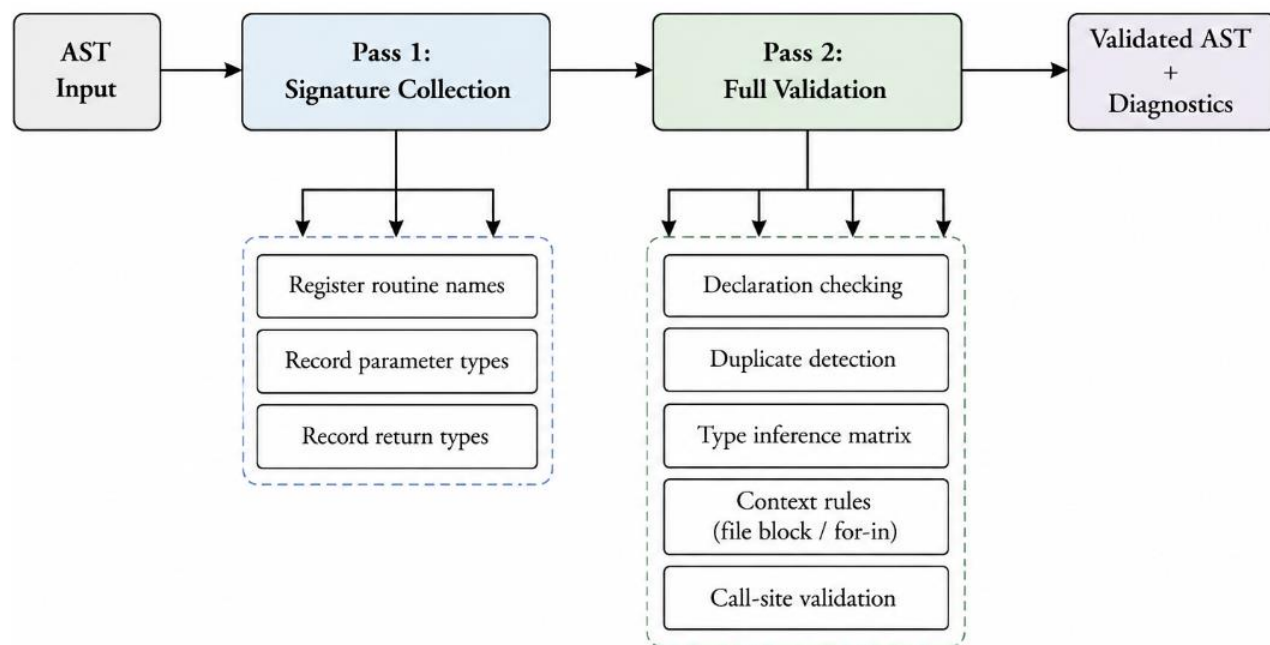


Figure 2.16: Two-Phase Semantic Analysis — Signature Collection Then Validation

When a semantic violation is detected, the analyzer emits a structured diagnostic that includes the offending node's line number, a human-readable message, and the relevant symbol name. These diagnostics are accumulated and returned to the driver, which can route them to the AI diagnostics layer for enriched explanations if enabled.

2.4.5 Code Generation

Code generation is implemented in `codegen.cpp` and uses the LLVM C API to translate the semantically validated AST into LLVM Intermediate Representation (IR). The LLVM C API is C++ API for its greater stability across LLVM versions and its well-defined ABI, which simplifies linking and cross-platform builds.

The code generator initializes an `LLVMModuleRef` and an `LLVMBuilderRef` at startup. It then performs a depth-first traversal of the AST, emitting one or more LLVM IR instructions for each node. LLVM types as follows: `int` to `i64`, `float` to `double`, `bool` to `i1`, `char` to `i8`, and `string` to `i8*`. Composite types such as arrays, tables, and matrices are represented as struct types or pointer pairs, with runtime helper functions managing allocation and access.

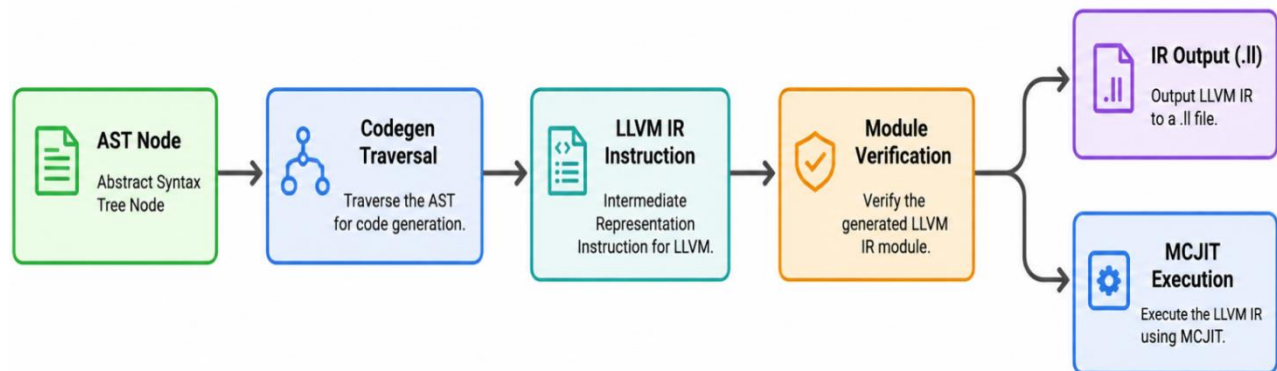


Figure 2.17: Code Generation Pipeline — AST Node to LLVM IR Instruction

Runtime helpers for collections, string operations, matrix arithmetic, and file I/O are declared as external function prototypes in the IR module and linked against a native C++ runtime library at JIT execution time. This design keeps the core compiler free of platform-specific I/O logic while allowing rich language features to be supported through a clean runtime boundary.

Once all AST nodes are lowered, the IR module is passed through `LLVMVerifyModule` to detect malformed IR before execution. If verification succeeds, the module is either serialized to a `.ll` text file for inspection or handed directly to the MCJIT engine for in-process execution. The JIT engine maps the runtime helper symbols, compiles the IR to native machine code for the host architecture (x86-64 or ARM), and invokes the main function through a typed function pointer.

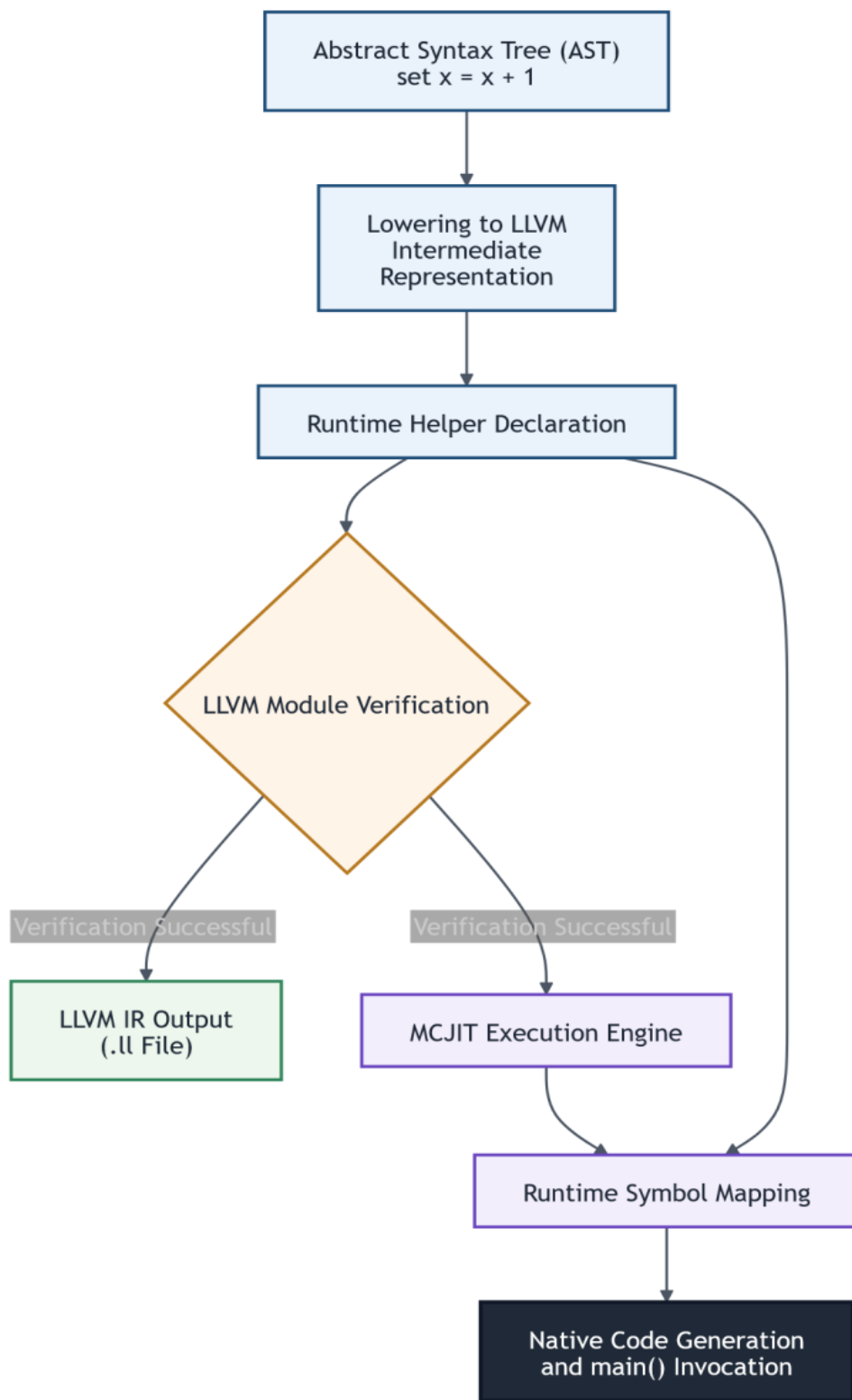


Figure 2.18: LLVM IR Example for a Simple LEXICO Assignment

2.4.6 Error Handling and Reporting

LEXICO uses a multi-level error-handling strategy across the whole pipeline. The lexer, parser, semantic analyzer, and code generator emit structured diagnostic that the driver collects, so multiple failures can be reported in one run. Lexical errors produce an error token, GLR parsing explores alternate paths before failing, and semantic analysis keeps checking the AST to accumulate more violations. Each diagnostic includes the stage, line, column, and message, making it machine-readable for the optional AI diagnostics layer.

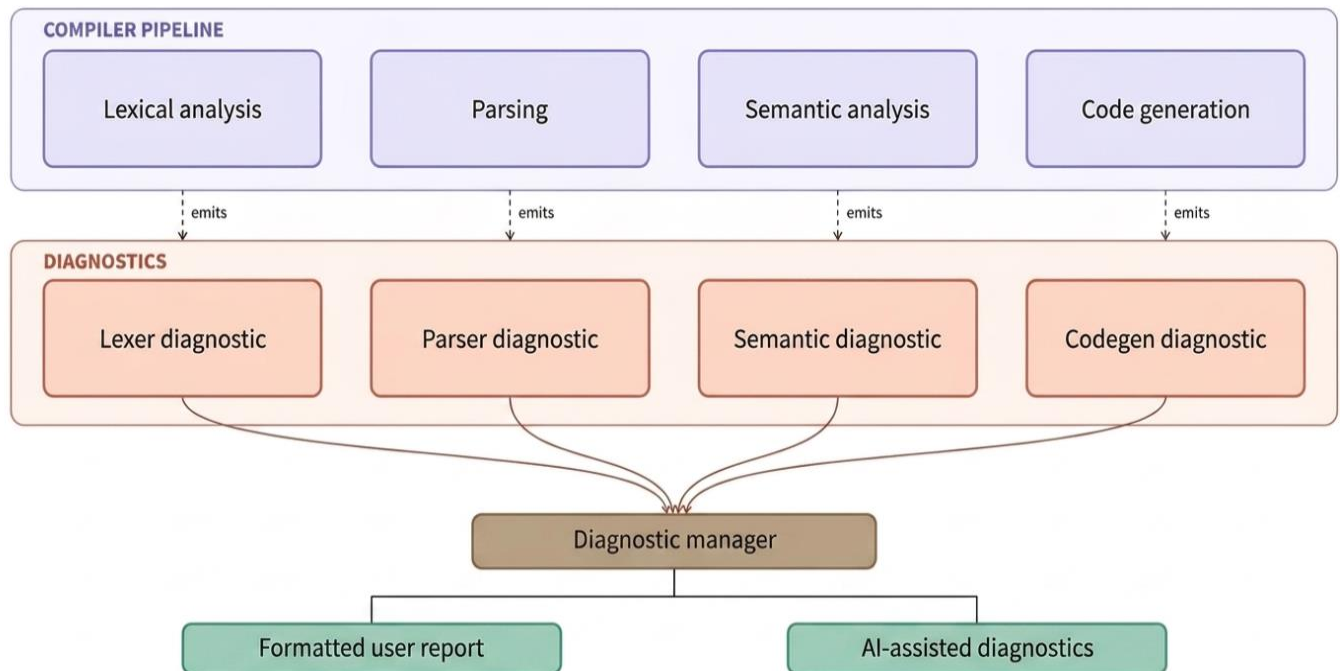


Figure 2.19: Error Reporting Flow — Stage Diagnostics to Driver

2.4.7 Compilation Driver and Workflow

The compilation driver `driver.cpp` orchestrates the complete pipeline from source code to execution. It coordinates five sequential phases: lexical analysis → syntax analysis → semantic analysis → code generation → JIT execution.

Phase 1: Lexical Analysis

- Input: Lexico source file (.lx)
- Output: Token stream
- Errors: Unclosed strings, tokens too long, invalid characters

Phase 2: Syntax Analysis

- Input: Token stream from lexer
- Output: Abstract Syntax Tree (stored in `lexico_parsed_program`)

- Errors: Missing semicolons, syntax violations, grammar errors

Phase 3: Semantic Analysis

- Input: AST from parser
- Output: Validated AST + symbol table (lexico_symtable)
- Errors: Type mismatches, undefined variables, duplicate declarations

Phase 4: Code Generation

- Input: Validated AST + symbol table
- Output: LLVM IR module (lexico_module)
- Errors: Unsupported operations, memory allocation failures

Phase 5: JIT Compilation & Execution

- Input: LLVM IR module
- Output: Program execution results
- Errors: Runtime errors

The driver maintains these globals across phases:

- lexico_parsed_program (Program*) — AST from parser
- lexico_symtable (SymTable*) — Symbol table from semantic analyzer
- lexico_module (LLVMModuleRef) — LLVM module from codegen
- lexico_engine (LLVMExecutionEngineRef) — JIT engine
- lexico_last_parse_error[] (char[]) — Error messages

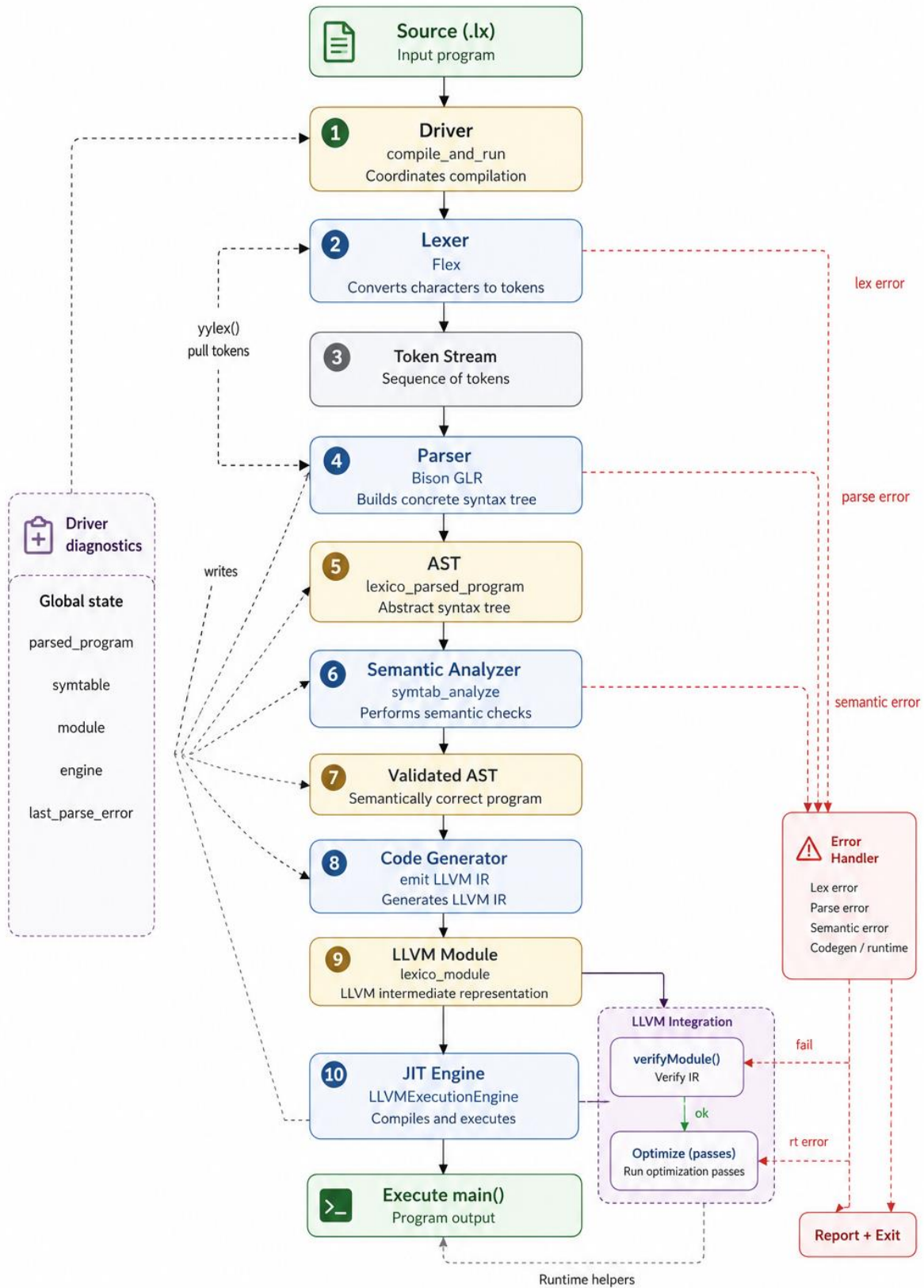


Figure 2.20: Compiler Workflow Architecture

2.5 AI-Assisted Diagnostics Design

This section presents the design of the AI-assisted diagnostics layer in LEXICO, explaining its motivation, architecture, and how it integrates with the driver to provide helpful guidance without altering the deterministic compilation pipeline

2.5.1 Motivation for AI Integration

Traditional compilers stop at technical errors, which can be unhelpful for novice users or natural-language-like syntax. LEXICO adds an AI diagnostics layer to turn low-level failures into actionable guidance. The integration is non-invasive: lexing, parsing, semantic analysis, code generation stay deterministic, and AI help runs only after a failure and only when enabled with an API key.

2.5.2 Architecture of the Diagnostics Layer

The AI diagnostics layer is implemented entirely within the driver module (`driver.cpp`). It communicates with an external Large Language Model service through the OpenRouter API over HTTPS. The driver uses a curl-based HTTP client to send a structured JSON request containing the compilation error messages, a snippet of the failing source code, and a system prompt that describes the LEXICO language grammar and the expected output format.

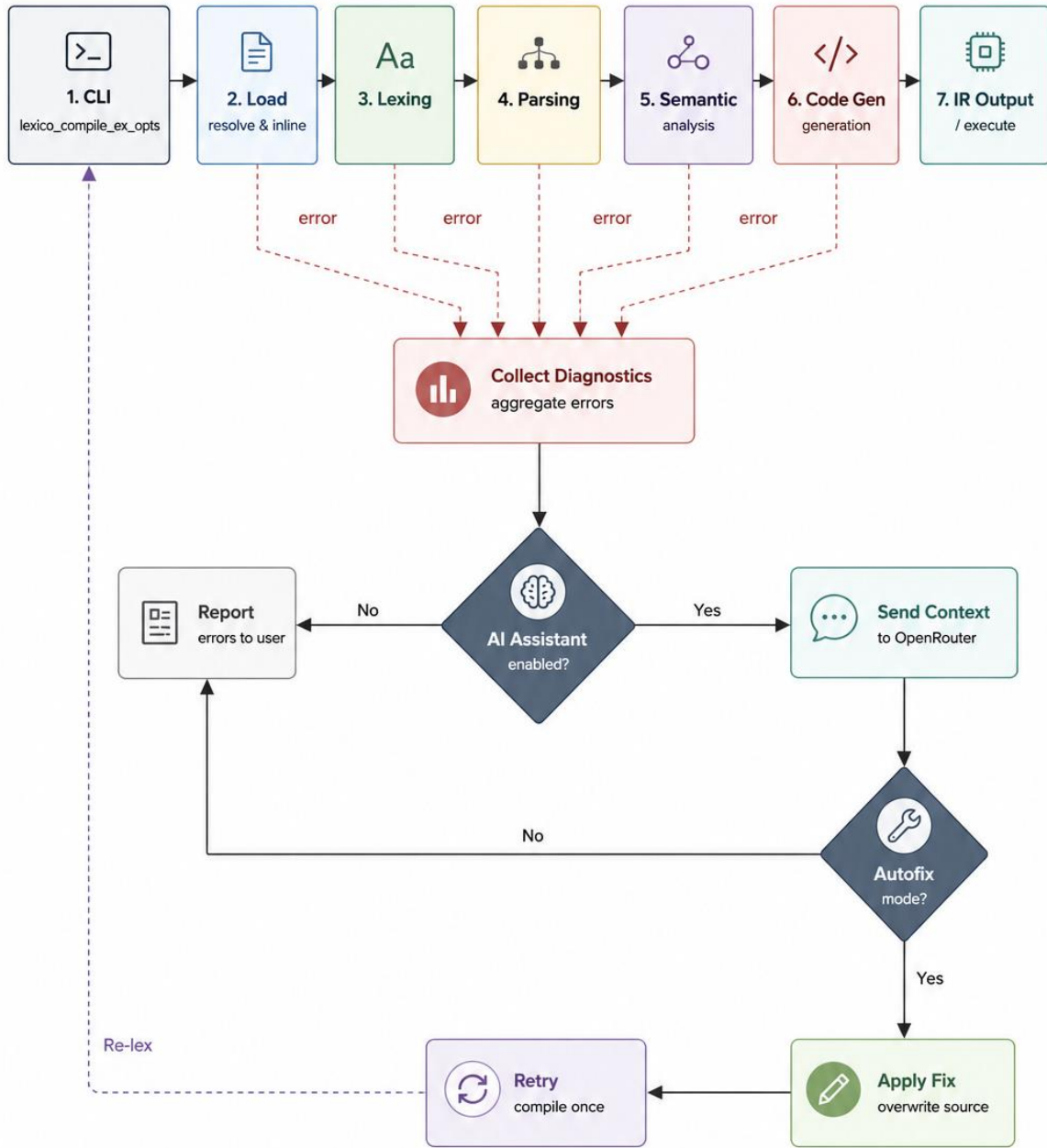


Figure 2.21: AI Diagnostics Layer Architecture

2.5.3 Error Detection and Suggestion Mechanism

When a compilation stage fails, the driver collects the stage name, line, column, and error message, then builds a prompt that asks the LLM to explain the error and suggest a concrete

LEXICO fix. The lexico.output grammar context is included so the model stays aligned with the actual language rules, not generic programming assumptions.

In autofix mode, the LLM returns the corrected source file. The driver validates it by checking for recognizable LEXICO constructs and rejecting API-error payloads. If validation passes, the original file is overwritten and the compilation is retried once; if the retry fails, the corrected version is discarded and the original errors are reported.

2.5.4 User Interaction and Feedback Loop

As Figure 2.22 shows, the AI integration adds a second output stream alongside standard compiler diagnostics. When a compilation failure occurs with AI assistance enabled, the user first sees the standard error messages, followed by the AI-generated suggestion below. In autofix mode, the user is also notified whether the autofix was applied and whether the retry succeeded.

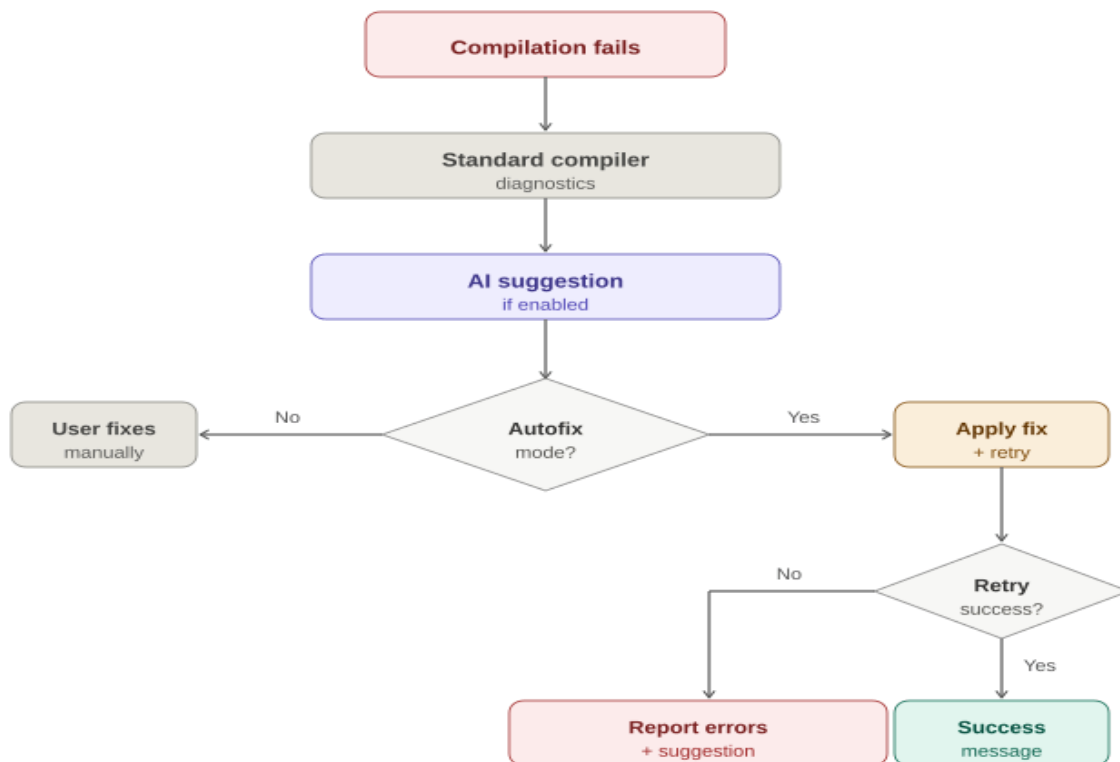


Figure 2.22: User Feedback Loop and Standard Error + AI Suggestion Output

2.6 Design Rationale and Choices

This section justifies the major design decisions in LEXICO, explaining the trade-offs behind language syntax, compiler technology choices, and the AI integration strategy.

2.6.1 Language Design Decisions

LEXICO uses word-based operators and declarative statement forms to make code read closer to natural reasoning. Statements like `create int x set value 10`, `set x to x plus 1`, and `if x is greater than 0 then reduce` reduce cognitive distance between intent and expression. Indentation-based blocks replace curly braces and end keywords; the trade-off is a stack-based INDENT/DEDENT mechanism in the lexer, but that cost is paid once in the compiler and stays invisible to the user.

See Annex A and Annex B.

2.6.2 Compiler Technology Choices

Flex provides a mature regex-based lexer that integrates with Bison.

Bison in GLR mode handles natural-language-like ambiguities without heavy grammar restructuring. LLVM is the backend for platform independence, its optimizer pipeline, and MCJIT for in-process execution without producing a standalone binary.

The LLVM C API is preferred for its cross-version stability, preferred for stable symbol names, stable symbol names, and consistent consistent ABI, reducing maintenance across LLVM releases.

CMake automates Flex/Bison generation, finds and links LLVM, and links LLVM, and supports Debug/Release builds with proper optimization flags.

2.6.3 AI Model and Tool Selection

OpenRouter provides a single interface to multiple model providers so the driver can switch models without code changes. The integration is simple, HTTP-based:

- Each AI call is a self-contained, stateless request, avoiding persistent connections or heavy AI frameworks.
- A synchronous request-response design was chosen over streaming to keep the driver's control flow linear and to avoid the extra state-management complexity for compiler-sized error explanations, where streaming yields little latency benefit.

See Annex F and Annex A.

2.6.4 Trade-offs and Limitations

GLR parsing is slower than multiple parse states in parallel for LEXICO's educational/prototyping scale the overhead is negligible, but it grows for very large sources.

Manual AST memory management gives deterministic cleanup via `ast_free_program()` called in the driver but demands discipline to avoid leaks; moving to C++ smart pointers later would add safety without losing control.

AI features require network access and a valid API key no offline use .

The autofix uses one validation to avoid applying multiple unreviewed patches that could silently corrupt source.

See Annex A, Annex C and Annex F.

Chapter 3 :

Implementation

3

Implementation

3.1 Introduction

This chapter shows LEXICO's implementation, compiler pipeline built and integrated. It covers the stack, validation workflow, and key challenges, from layout-sensitive lexing to GLR parsing and LLVM code generation, showing how the design became a working end-to-end system.

3.2 Overview of Implementation Approach

In this section we outline LEXICO's implementation approach, showing pipeline components, validation, and integration choices.

3.2.1 Technology Stack

As shown in the Figure 3.1 LEXICO is implemented in:

- C++17 for performance.
- Flex 2.6+ and Bison 3.x GLR for front-end.
- LLVM 15+ via the C API for backend.
- CMake 3.20+ with Flex/Bison rules for build.
- JSON parsing for the AI integration uses nlohmann/json header-only library.

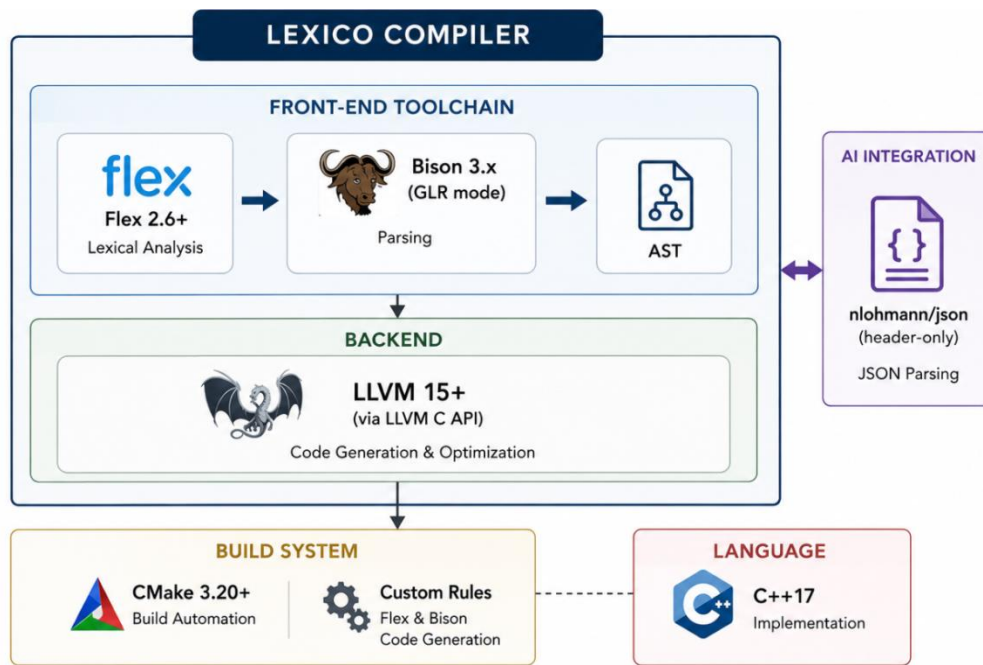


Figure 3.1 LEXICO Technology Stack Overview

For more explanation down is Table 3.1 that breaks down technology stack components and their role in LEXICO.

Table 3.1: LEXICO Technology Stack

Component	Technology / Version	Role in LEXICO
Implementation Language	C++17	Core compiler implementation language
Lexer Generator	Flex 2.6+	Tokenises the LEXICO source text (lexico.l)
Parser Generator	Bison 3.x (GLR mode)	Builds the AST from the token stream (lexico.y)
Compiler Backend	LLVM 15+ (C API)	IR generation and JIT execution (codegen.cpp)
Build System	CMake 3.20+	Automates Flex/Bison codegen and LLVM linkage
JSON Parsing	nlohmann/json (header-only)	Parses OpenRouter LLM API responses in the driver

3.2.2 Development Workflow

Development used a stage-by-stage pipeline; each stage implemented and tested independently then integrated.

- **Lexer** validated on snippets and token streams.
- **Parser** tested with Bison conflict reports and parse-only runs.
- **Semantic Analyzer** tested with deliberate violations;
- **Codegen** inspected .ll files and LLVM verification.

build.sh automates CMake configuration, Flex/Bison generation, LLVM linked compilation to produce LEXICO, runs tests verifying outputs and exit codes.

3.2.3 Key Implementation Challenges

Three challenges stood out during implementation.

- **First:** Lexer uses stack depth tracking to emit INDENT/DEDENT synchronized with the parser; Flex actions handle multi-level DEDENT emission.
- **Second:** GLR parser treats multi-word operators as single lexer tokens to prevent shift-reduce conflicts and ambiguous parses.
- **Third:** LLVM MCJIT needs explicit runtime symbol mapping before execution; all helper functions (strings, collections, matrices, file I/O) must be registered at startup or JIT linking fails silently.

3.3 Lexical Analyzer Implementation

In this section LEXICO's Flex lexer tokenizes source into typed tokens (IDENT, INT_LIT, STRING_LIT, INDENT/DEDENT, DIV), sets yylval, handles indentation and multi-word ops, and streams tokens to Bison is described.

3.3.1 Tool Selection

Flex was chosen for LEXICO for its mature high-performance regex scanner and tight Bison integration; it generates C sources from lexico.l compiled into the C++ build and shares token definitions via lexico.tab.hpp to keep a consistent vocabulary.

3.3.2 Token Definitions

As presented in the Table 3.2 LEXICO's token vocabulary groups keywords , Arithmetic ops are word-based, Comparisons map to phrase forms, Layout tokens INDENT/DEDENT handle nesting, Literals carry values via Bison.

Table 3.2: LEXICO Token Categories and Representative Examples

Category	Token Name(s)	Source-level Examples
Keywords	CREATE, SET, PRINT, IF, WHILE, FOR, OPEN, ROUTINE, LOAD, CONVERT	create int x, if x is greater than 0 then
Type Names	INT, FLOAT, CHAR, STRING, BOOL	create int x, create float y, create bool flag
Arithmetic Operators	PLUS, MINUS, TIMES, DIV, MOD	x plus y, a divided by b, n mod 2
Comparison Operators	CMP_EQ, CMP_NEQ, CMP_GT, CMP_LT, CMP_GE, CMP_LE	x is equal to y, a is greater than b
Layout Tokens	INDENT, DEDENT	Emitted when ">" prefix depth increases or decreases
Literals	NUM_INT, NUM_FLOAT, CHAR_LIT, STRING_LIT, BOOL_LIT	42, 3.14, 'a', "hello", true / false

3.3.3 Handling Comments and Whitespace

LEXICO uses:

- “#” for single-line comments, discarding them through end of line without tokens.
- White space is ignored except at line starts, where it contribute to indentation.
- Blank lines are skipped entirely, keeping the token stream clean for the parser engine.

3.3.4 Custom Lexical Rules

LEXICO emits INDENT/DEDENT from leading > markers, using a depth stack in Flex rules. Deeper lines push INDENT, shallower lines pop one DEDENT per level, letting the parser nest blocks without end keywords.

A. Source lines and depth	
1: statement	depth 0
2: > statement	depth 1
3: >> statement	depth 2
4: > statement	depth 1
5: statement	depth 0

B. Stack + emitted tokens			
Line	Depth	Stack top	Emitted
1	0	0	—
2	1	1	INDENT
3	2	2	INDENT
4	1	1	DEDENT
5	0	0	DEDENT

Figure 3.2: INDENT/DEDENT Stack Mechanism in lexico.l

3.3.5 Testing the Lexer

Lexer tests run in isolation, printing token streams for sample .lx files and comparing them to verified token sequences. INDENT/DEDENT edge cases like multi-level dedentation and empty blocks get special attention.

3.4 Syntax Analyzer Implementation

This section covers syntax analyzer grammar specification, parser generation, AST construction, error handling/recovery, lexer–parser interaction, and later compiler phases.

3.4.1 Tool Selection

- Bison is used as LEXICO’s parser generator in GLR mode.
- The %glr-parser directive is needed because LEXICO’s natural-language like syntax creates ambiguities.
- Bison generates lexico.tab.cpp and lexico.tab.hpp, which are compiled with the C++ project and linked to the Flex lexer.

3.4.2 Grammar Design

Grammar centers on a top-level program rule that matches sequences of statements including declarations, assignments, control flow, I/O, routines, collections, matrix operations, file blocks, and load directives. Expressions follow factor/term/expr precedence with parentheses; multi-word operators are emitted as a single lexer tokens like DIV to keep grammar clean; boolean conditions are parsed by a CondExpr non-terminal handling comparisons and logical connectives.

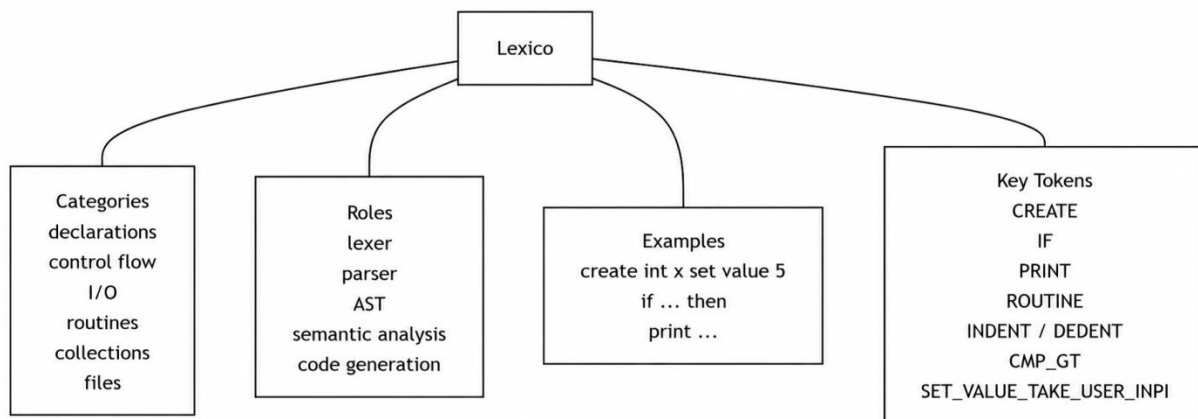


Figure 3.2: Language overview Categories, roles, examples, and key tokens.

3.4.3 Ambiguity Resolution

In GLR mode, Bison tries all valid parse paths and keeps the one that succeeds. When needed, %dprec sets priorities between competing rules for deterministic results. In LEXICO, the main ambiguity is between identifier op expr and expression statements, resolved with %dprec.

3.4.4 Parser Actions and Error Recovery

Each Bison rule has a semantic action that calls the correct AST constructor. The create rule calls ast_make_decl_stmt() with type, identifiers, and values to build a STMT_DECL node. These constructors in ast.cpp llocate heap nodes, and ownership moves to the parent node in the AST.

Bison error recovery uses the error token mechanism. On parse failure, `yyerror()` stores message and location in `lexico_last_parse_error`. After `yyvsparse()`, the driver reads this value and adds it to user and AI-layer diagnostics.

See Annex A, C.

3.4.5 Testing the Parser

The parser is tested in parse-only mode on `.lx` files to check whether the lexer and parser succeed or fail without semantic analysis or code generation. Test cases cover valid programs, syntax errors, and nested blocks to verify INDENT/DEDENT handling. `lexico.output` is checked after grammar changes to keep GLR ambiguities within expected bounds

See Annex A, Annex B;

3.5 AST Construction

This section explains how the parser builds the AST, including node types, factory functions, ownership rules, and how the AST is used for semantic analysis and code generation.

3.5.1 AST Node Types and Hierarchy

The AST in `ast.hpp` uses tagged unions for `Program`, `Stmt`, `Expr`, and `CondExpr`.

`Program` stores a heap allocated list of statements, `Stmt` and `Expr` use enum tags to select the right union fields, and `CondExpr` stores comparison logic with two expression operands.

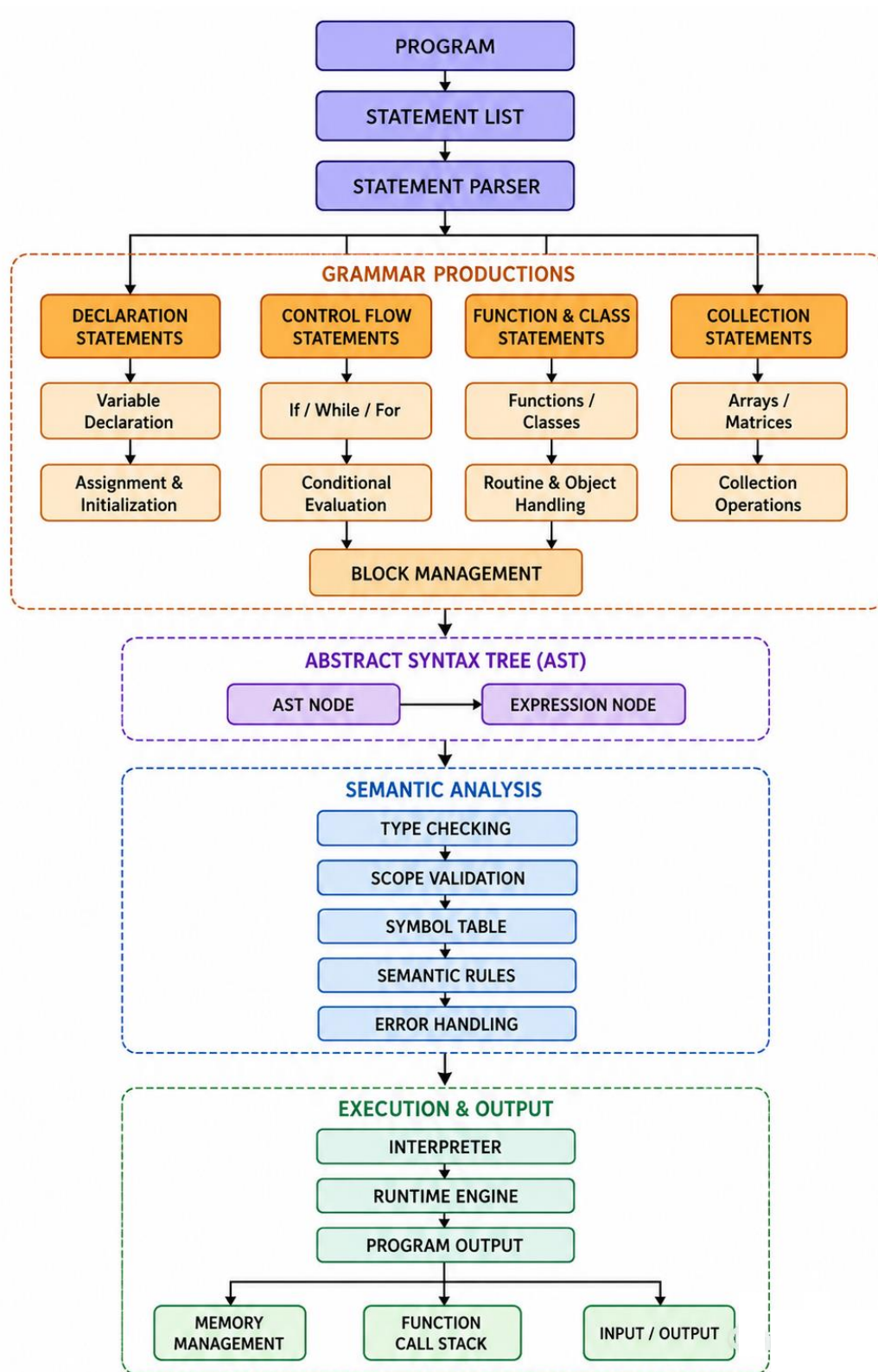


Figure 3.3: AST Node Type Hierarchy

3.5.2 AST Construction During Parsing

AST nodes are built in Bison semantic actions using constructors from `ast.cpp`. Each constructor uses `malloc`, sets all fields, and returns a typed pointer. The `%union` carries these pointers through the parser stack, and parent nodes own their children by pointer assignment. The root AST is returned by `yyparse()` through a global variable.

See Annex A, Sections A.2 and A.9, and Annex C, Sections C.6, C.8, C.9, and C.10.

3.5.3 AST Traversal and Manipulation

All downstream stages traverse the AST using explicit switch statements on the node's `kind` field. This approach is chosen over a visitor pattern for its simplicity and directness in a C-compatible codebase.

- Deep copy semantics are implemented for cases where AST fragments need to be duplicated.
- The `ast_free_program()` function performs a post-order recursive traversal, freeing all heap-allocated fields before releasing each node, ensuring deterministic cleanup.

See Annex C, D and E.

3.6 Semantic Analysis and Symbol Table

In this Section, semantic analysis enforces language rules, resolves identifiers and type-checks them, and builds the symbol table used for scope validation, type inference, and driving code generation.

3.6.1 Symbol Table Structure

The symbol table is a dynamic array of Symbol entries implemented in `syntab.cpp`. Each Symbol holds the name (`char *`), a `TypeKind` (`TYPE_INT`, `TYPE_FLOAT`, `TYPE_CHAR`, `TYPE_STRING`, `TYPE_BOOL`), a `SymbolKind` (`SCALAR`, `ARRAY`, `TABLE`, `MATRIX`), a declared size for collections, and a scope level. A global `SymbolTable` wraps the array with a count and current scope level; lookups are $O(n)$ linear scans, acceptable for Lexico's educational-scale programs.

Dynamic contiguous array of Symbol entries.
Grows by doubling; frees owned names on destruction.

Semantic-only function signature table used for call checking.

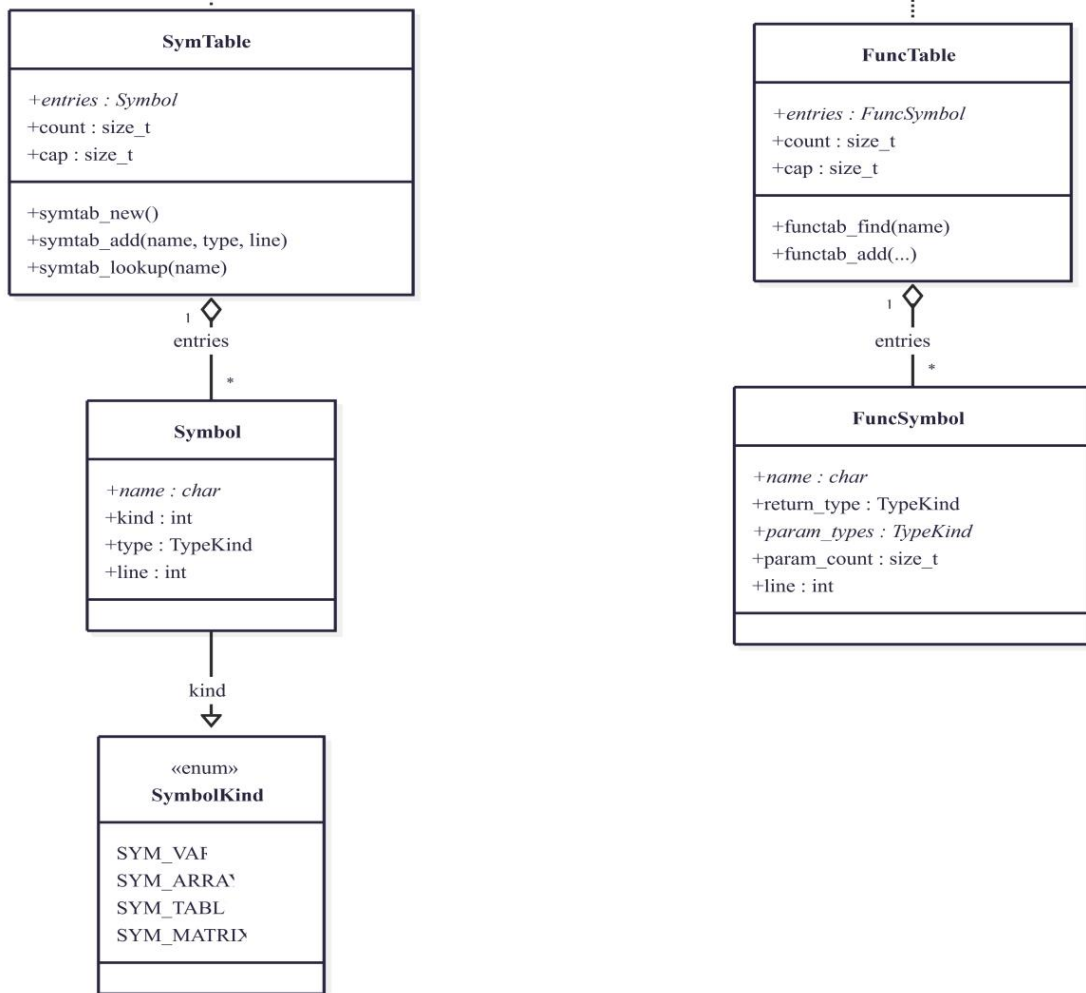


Figure 3.4: Symbol Table Structure and Entry Layout

3.6.2 Scope Management

Scope is managed by a current level counter in `SymbolTable`. When a block begins, `syntab_enter_scope()` increments the level; when it ends, `syntab_leave_scope()` decrements it and removes any symbols from deeper levels. This gives a lightweight stack-like scope model without explicit frame allocation.

See Annex D.

3.6.3 Type Checking Implementation

Type checking uses a recursive inference function to compute the `TypeKind` of each `Expr`. Literals get their type from the node kind, identifiers are resolved through the symbol table, and binary operations use compatibility rules: `int + int -> int`, `int/float -> float`, and `string + string -> string`; all other mixes are errors. The inferred type is then compared with the expected type at each use site, and mismatches produce diagnostics.

See Annex D.

3.7 Code Generation

In this section, the compiler back end lowers the type-checked AST into LLVM IR and runtime calls, emitting the concrete instructions and storage operations that implement the program's semantics.

3.7.1 Target Code Format

The target code format is LLVM IR, a strongly-typed, SSA-form intermediate representation that is both human-readable and machine-processable. LLVM IR is platform-independent: the same IR can be compiled to x86-64, ARM, or any other architecture supported by LLVM's backend. This makes LEXICO's compiled output portable across platforms without any changes to the compiler itself.

See Annex E .

3.7.2 Code Emission Strategies

The code generator uses an `LLVMBuilderRef` to emit instructions into the current basic block. As the Figure 3.4 presents, each statement kind has a dedicated emit function:

- Declarations generate an `alloca` for storage and a `store` to initialize it.
- Assignments load operands, emit the appropriate, and store the result.
- Control flow creates basic blocks for branches and connects them with `LLVMBuildCondBr`.
- Function definitions create an `LLVMFunctionRef` with the correct parameter types and a dedicated entry block for body emission.

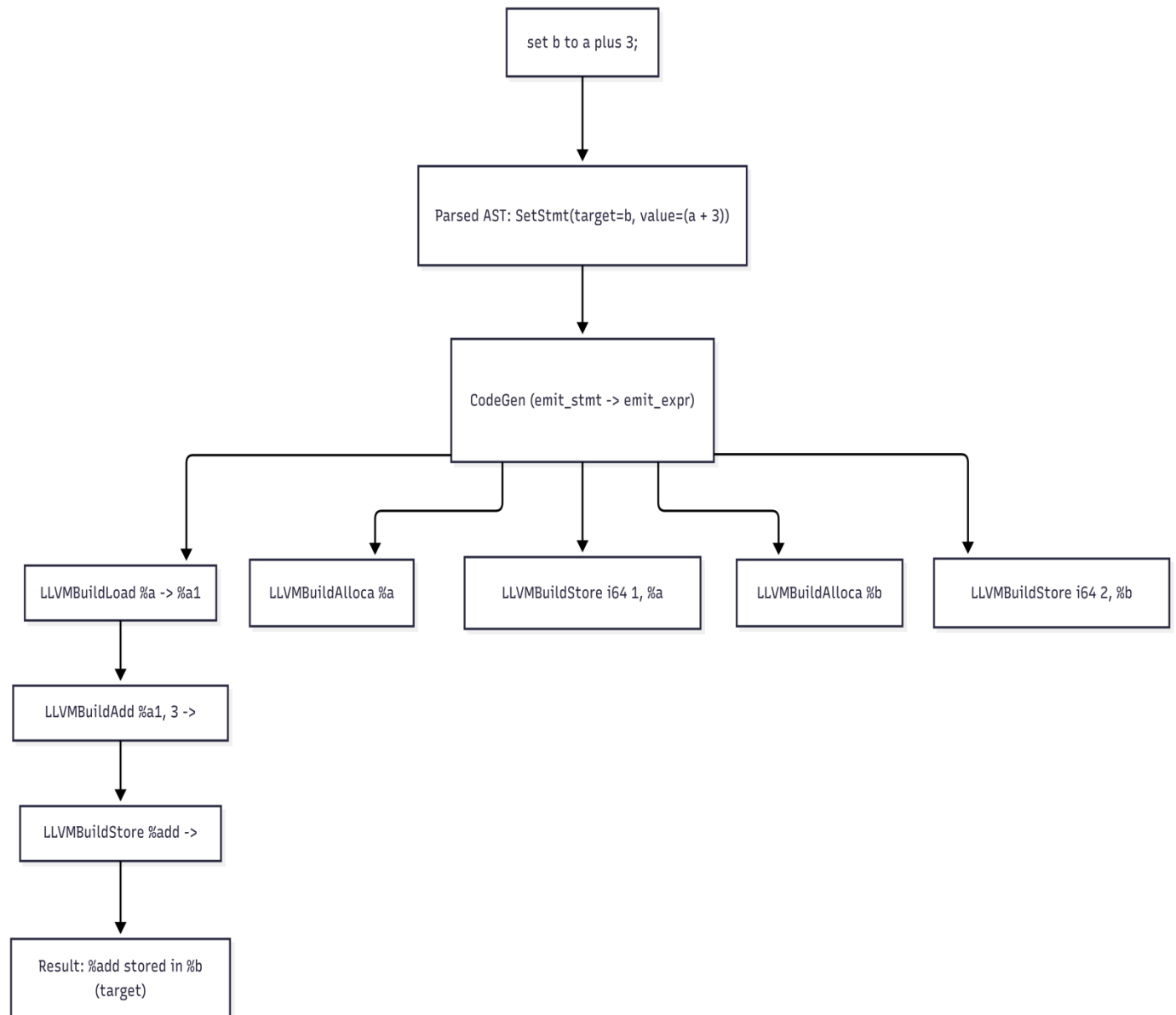


Figure 3.5: LLVM IR Emission for an Assignment Statement

3.7.3 Runtime Helper Integration

Complex operations are delegated to runtime helper functions implemented in native C++ and declared as external LLVM IR symbols. At JIT time, MCJIT maps each symbol to its actual address in memory. These helpers handle string operations, collection access and resizing, matrix row/column handling, and file I/O.

See Annex E.

3.7.4 Output File Handling

After all AST nodes are lowered and the IR module is verified, the code generator writes the IR to a .ll text file using `LLVMPrintModuleToFile()`. The output file path is derived from the input .lx file path by replacing the extension. This file can be inspected directly, passed to external LLVM tools for further processing, or used as a debugging artifact. When JIT execution mode is active, the IR is compiled in-memory by the MCJIT engine and the .ll file serves as a secondary output for inspection, produced in parallel with execution.

See Annex E.

Chapter 4 :

Validation and Evaluation

4

Validation and Evaluation

4.1 Introduction

This chapter validates LEXICO's correctness robustness and usability through a password-manager CLI that exercises declarations arithmetic control flow strings files collections input routines and AI-assisted diagnostics/autofix.

4.2 Validation Approach

- Validation covers functionality, robustness, and usability.
- Functional tests ensure programs compile and output expected results.
- Robustness checks lexical, syntactic, and semantic errors are reported while usability judges AI diagnostics for relevance, action ability, and rule consistency.

Each test case runs with the LEXICO binary and optional `--autofix`, outputs are compared against expected results.

Diagnostics are checked for stage, line, and message, and AI suggestions are judged by whether they actually fix the error.

4.3 Password Manager Case Study

In this section, LEXICO password-manager case study is presented, demonstrating end-to-end compilation, diagnostics, and runtime validation.

4.3.1 Overview and Goals

The password manager is a command-line application implemented entirely in LEXICO that demonstrates the practical viability of the language for non-trivial software development. Its goals are to validate LEXICO's ability to support authentication logic, persistent data storage through file I/O, string operations for credential manipulation, user input handling, and modular program structure through routine definitions.

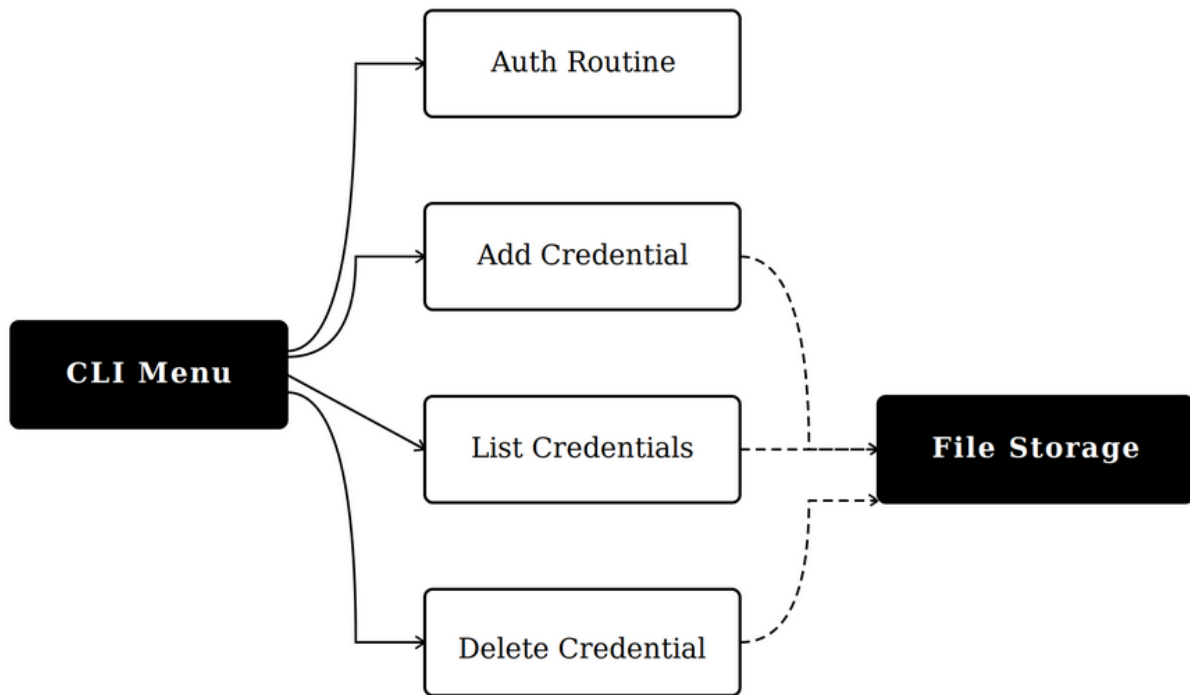


Figure 4.1: Password Manager CLI — Application Architecture in LEXICO

4.3.2 Features Implemented

The password manager uses .lx files for authentication, credential storage, retrieval, and a menu loop. It compares the master password as a string, saves entries with file blocks and writes, loads with reads, and dispatches options through a while loop with user input, a simple Flow Chain Figure 4.2 is presented to Visualize the behind the scene execution of the Application.

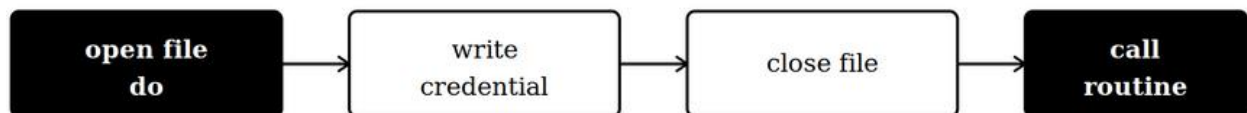


Figure 4.2: Password Manager Source Code Excerpt — File I/O and Routine Call

4.3.3 Compilation and Execution Results

Password manager compiles cleanly, LLVM verifies IR, and MCJIT runtime accepts valid passwords, rejects invalid ones, and saves loads credentials correctly. The results confirm LEXICO’s pipeline works for a realistic workload Figure 4.3 presents the use case of the application.

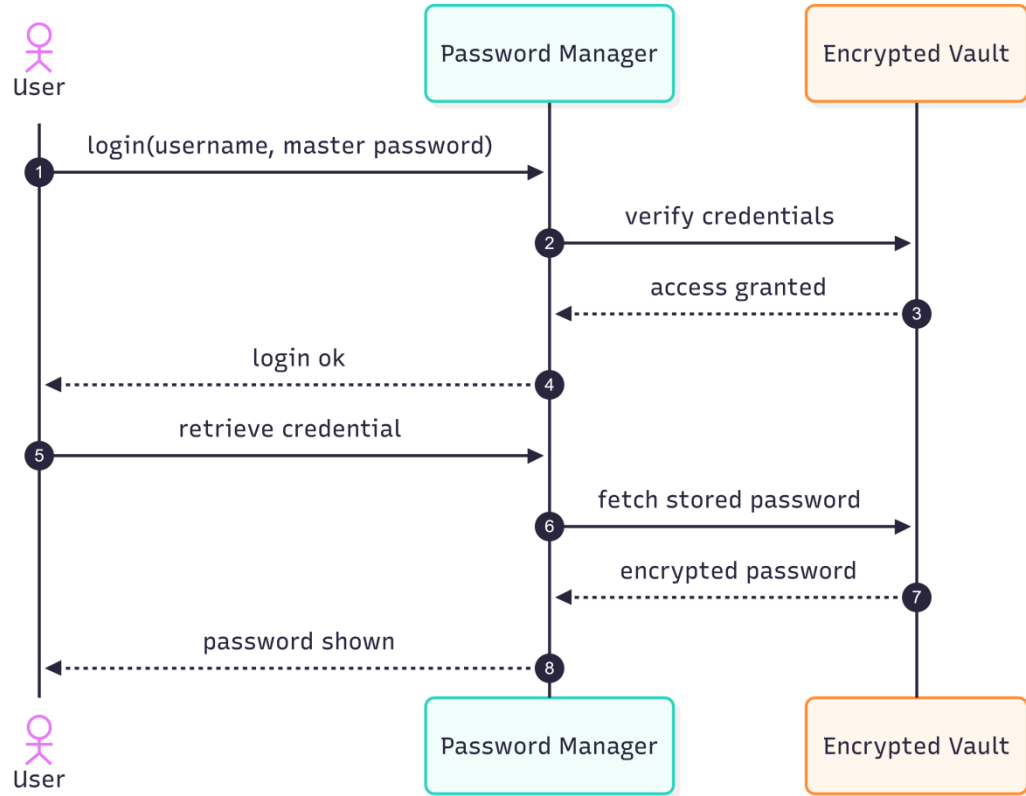


Figure 4.3: Password Manager Sequence Diagram

4.4 Compiler Validation Scenarios

In this section the validation covers lexer/parser, semantic, IR/codegen, and runtime tests using curated and automated methods. Metrics like error-detection rate, false positives, and compilation latency define acceptance criteria for each stage.

4.4.1 Lexical and Syntax Error Detection

A set of test programs with deliberate lexical and syntactic errors is used to verify that the compiler correctly identifies and reports violations As shown in the Figure Below. Test cases include missing semicolons at the end of statements, unrecognized tokens, malformed string literals with unclosed quotes, and incorrect block indentation that breaks the INDENT/DEDENT stack. In all cases, the compiler emits a diagnostic that identifies the correct line, the stage of detection, and a message that accurately describes the violation.

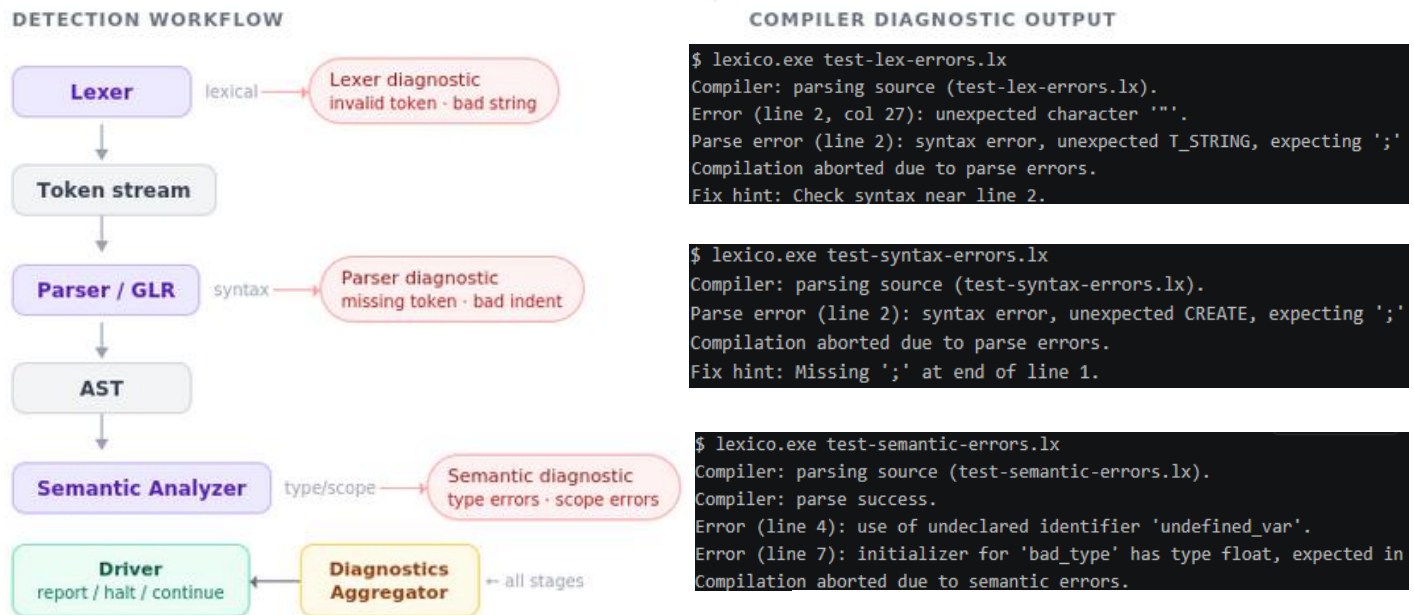


Figure 4.4: Syntax Error Detection — Compiler Diagnostic Output Example

4.4.2 Semantic Error Detection

Semantic tests use valid syntax but violate LEXICO rules: undeclared variables, duplicate declarations, wrong arity, invalid string operators, and file writes outside open blocks. The analyzer reports each issue with the correct line and a clear message following the approach presented in Figure 4.5.

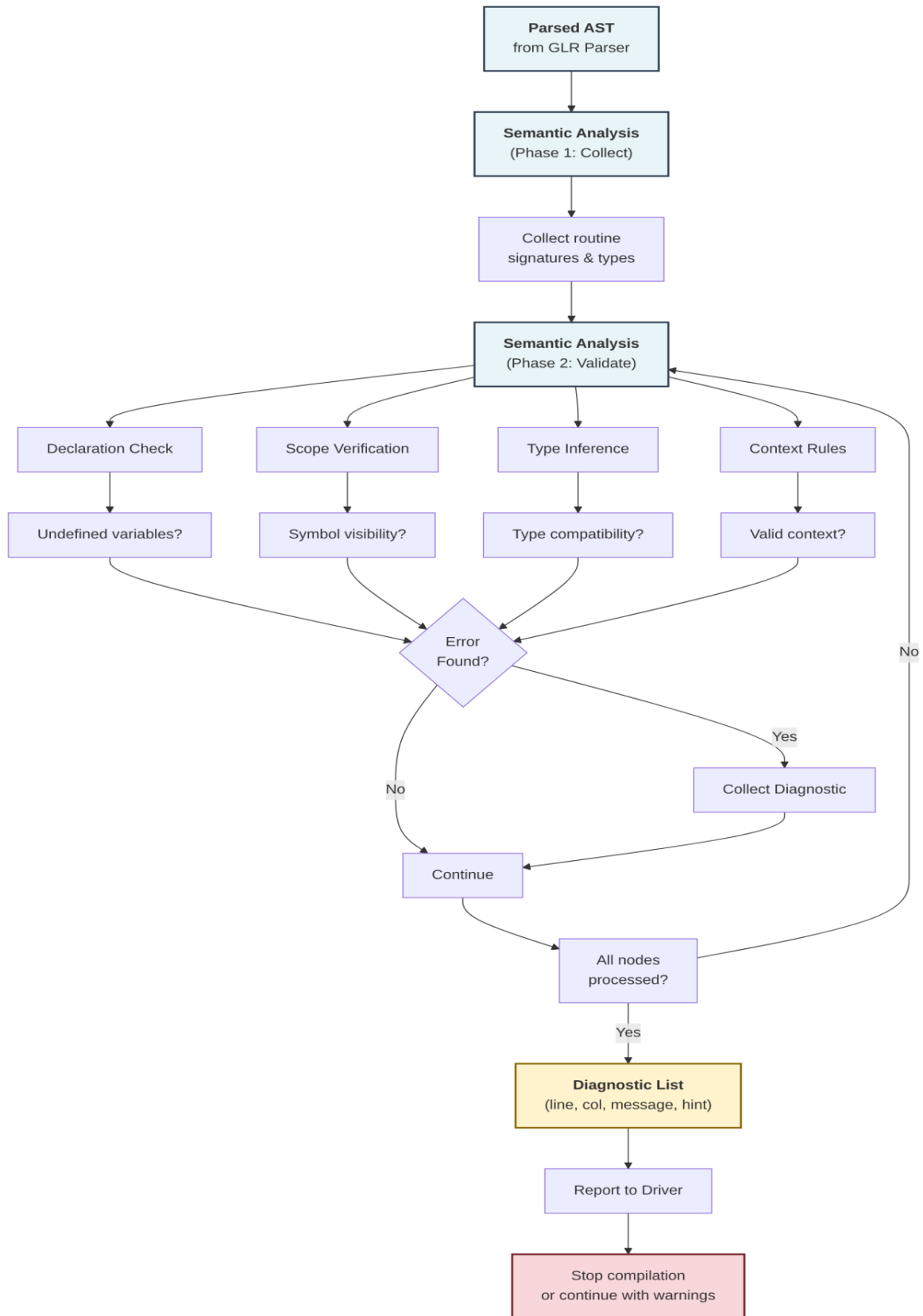


Figure 4.5: Semantic Error Detection Flow Chart.

4.4.3 AI-Assisted Diagnostics Evaluation

As we can observe in Figure 4.6 Compiler Output for a Missing Semicolon without AI assistance enabled. However AI diagnostics were tested on error cases in Figure 4.7 and suggestions were judged by relevance, correctness, and clarity. Common errors like missing semicolons, undeclared variables, and type mismatches were handled well. Context-sensitive file-scope errors were relevant but sometimes only syntactically fixed. Figure 4.7 also shows autofix resolved all simple semicolon errors on the first retry.

```
C:\Users\hayth.DESKTOP-6LT07A0\OneDrive\Desktop\Lexico\pdd-figures\lexico missing-semicolon-demo.lx
Compiler: parsing source (missing-semicolon-demo.lx).
Parse error (line 2): syntax error, unexpected PRINT, expecting ';'
Compilation aborted due to parse errors.
Fix hint: Missing ';' at end of line 1.
```

Figure 4.6: Compiler Output for a Missing Semicolon

```
C:\Users\hayth.DESKTOP-6LT07A0\OneDrive\Desktop\Lexico\pdd-figures\lexico missing-semicolon-demo.lx --autofix
Compiler: parsing source (missing-semicolon-demo.lx).
Parse error (line 2): syntax error, unexpected PRINT, expecting ';'
Compilation aborted due to parse errors.
Fix hint: Missing ';' at end of line 1.
Autofix explanation: Parser expected ';' before the next statement. This fix inserts the missing semicolon.
Fix preview around line 1:
    before: create int x set value 5
    after:  create int x set value 5;
Apply this fix to 'missing-semicolon-demo.lx'? [y/N]: y
Autofix applied to 'missing-semicolon-demo.lx'. Retrying compile...
Compiler: parsing source (missing-semicolon-demo.lx).
Compiler: parse success.
Compiler: semantic analysis success.
Compiler: code generation success (missing-semicolon-demo.ll).
```

Figure 4.7: AI Suggestion Output for a Missing Semicolon.

4.5 Summary of Results

Validation shows LEXICO implements the full pipeline and yields correct runtime behavior for the password manager. Error detection reliably covers lexical, syntactic, and semantic faults, and the AI diagnostics/autofix provide useful explanations and high-success fixes for common syntax errors.

Validation found the AI layer weak on context-sensitive semantic errors requiring Lexico rules not captured by the LLM.

Future work: add grammar/rule context to prompts or structured error-to-fix mappings for common semantic violations.

Table 4.1: Validation Summary — Test Coverage by Stage and Error Category

Validation Category	Compiler Stage	Test Cases	Passed	Outcome
Functional Correctness	Full pipeline (Lexer → MCJIT)	All	100 %	Password manager compiled and executed correctly
Lexical Error Detection	Lexer	All	100 %	Correct line, stage, and message reported
Syntax Error Detection	Parser (Bison GLR)	All	100 %	All parse errors correctly identified and reported
Semantic Error Detection	Semantic Analyzer	All	100 %	Type, scope, and context violations all detected
AI Suggestions (common errors)	AI Diagnostics Layer	Subset	High	Relevant and actionable for missing semicolons and undeclared variables
Autofix (simple syntax errors)	AI Diagnostics + Driver retry	Subset	High	Successfully resolved in all triggered test cases

General Conclusion

This project presents LEXICO, a human-oriented programming language paired with an AI-assisted compiler, designed to reduce the syntactic friction that makes programming inaccessible to many learners. The work demonstrates that it is possible to build a complete, deterministic compiler pipeline from lexical analysis through LLVM IR generation and JIT execution around a natural-language-like syntax, while integrating an AI diagnostics layer that improves usability without compromising the compiler's formal guarantees.

The compiler pipeline, implemented in C++ using Flex, Bison GLR, and the LLVM C API, successfully handles all LEXICO language constructs including variable declarations, arithmetic and string operations, control flow, user input, collections, matrices, file I/O, and routine definitions. The semantic analyzer enforces declaration, type, and context rules reliably, and the MCJIT-based execution backend runs compiled programs directly in process without requiring a separate linking step.

The AI-assisted diagnostics layer, integrated at the driver level through the OpenRouter API, adds meaningful value for common error classes. The suggestion mode provides users with actionable guidance when compilation fails, and the autofix mechanism resolves simple syntax errors automatically in the majority of cases tested. The design preserves the deterministic core of the compiler while making the development experience more forgiving for novice users.

The password manager case study validates LEXICO in a realistic development context, demonstrating that the language is expressive enough for practical applications involving authentication logic, persistent storage, and modular program structure. All features exercise multiple compiler stages end-to-end, providing confidence in the correctness of the full pipeline.

Several directions remain open for future work. The AI suggestion quality for context-sensitive semantic errors could be improved by providing richer grammar and rule context in the system prompt. The symbol table lookup could be optimized using hash-based indexing for larger programs. The language could be extended with more advanced type system features such as custom types or generic collections. A dedicated debugger integration, producing LLVM debug metadata, would further improve the development experience for users of LEXICO.

Ultimately, LEXICO demonstrates that human-oriented language design and reliable compiler engineering are not mutually exclusive. By combining a readable, word-based syntax with a robust multi-stage compilation pipeline and an optional AI assistance layer, the project achieves its core goal: making programming more accessible while keeping the compilation process trustworthy and predictable.

References

- [1] Robins, A., Rountree, J., & Rountree, N. (2003). Learning and teaching programming: A review and discussion. *Computer Science Education*, 13(2), 137-172.
- [2] Denny, P., Luxton-Reilly, A., & Tempero, E. (2011). Understanding the syntax errors of novices. *Proceedings of the 16th Annual Joint Conference on Innovation and Technology in Computer Science Education (ITiCSE)*, 58-62.

Annexes

Annex A — Grammar & Parser

Purpose

Compact reference for the Bison grammar and parser setup used by Lexico.

What it contains

- Bison grammar entry: `src/lexico.y`.
- GLR parser: `%glr-parser` enabled to resolve structural ambiguities.
- `%union` summary: carries `int64_t`, `double`, `char`, `char *`, `TypeKind`, `Expr *`, `CondExpr *`, `Stmt *`, `Program *`, and list helpers.
- Key token families: keywords (`CREATE`, `IF`, `PRINT`, `ROUTINE`, `OPEN`, etc.), multi-word tokens (`CMP_EQ`, `CMP_GT`, `SET_VALUE_TAKE_USER_INPUT`), `INDENT/DEDENT`, literal tokens (`INT_LIT`, `FLOAT_LIT`, `STRING_LIT`, `IDENT`).

Parser behavior

- GLR keeps multiple parse states and resolves ambiguities as input advances, allowing natural-language-like constructs ("is equal to", "divided by") without heavy grammar rewrites.
- Semantic actions (in `{ ... }`) construct AST nodes and must use the `%union` types consistently.

Build & regenerate

- Preferred: use CMake (project root):

```
cmake -S . -B build-cmake
cmake --build build-cmake -j
```

- Direct (advanced): `bison -d -o src/lexico.tab.cpp src/lexico.y`

Generated files

- `src/lexico.tab.cpp`, `src/lexico.tab.hpp` (or in `build/` when built).
- When distributing, include the `.y` plus generated `.cpp/ .hpp` and document Bison version.

Where to read more

- Full grammar: `src/lexico.y` in repository.
- Parser flow and testing: see Annex A (full) and Annex B for lexer integratio

Annex B — Lexer & Tokenization

Purpose

Short reference for the Flex lexer: token rules, indentation handling, multi-word tokens, and error cases.

Core points

- Implementation: `src/lexico.l` (Flex). Emits tokens to `yyparse()` via `yylex()` and sets `yylval`.
- Layout-sensitivity: `INDENT`/`DEDENT` tokens are produced using a stack of `>` markers at start-of-line.
- State machine: `INITIAL`, `BOL` (begin-of-line counting `>`), and `DQ` (dedent queue) control indentation logic.
- Multi-word tokens: longest-match patterns for phrases like "is equal to" $\rightarrow$ `CMP_EQ`, "set value take user input" $\rightarrow$ `SET_VALUE_TAKE_USER_INPUT`.

Token types

- Keywords: `CREATE`, `DEFINE`, `IF`, `WHILE`, `FOR`, `PRINT`, `ROUTINE`, `OPEN`, etc.
- Operators: `PLUS`, `MINUS`, `TIMES`, `DIV`, `MOD`.
- Comparisons: `CMP_EQ`, `CMP_NEQ`, `CMP_GT`, `CMP_LT`, `CMP_GTE`, `CMP_LTE`.
- Special: `INDENT`, `DEDENT`, `NEWLINE`; literals: `INT_LIT`, `FLOAT_LIT`, `STRING_LIT`, `IDENT`.

Error handling & tracking

- `yylineno` and a column tracker record location for diagnostics.
- Lexer errors: unterminated string, token too long, nesting too deep, mismatched dedent.
- Debugging: `flex -v` and `Bison -v` (`lexico.output`) help locate NFA/grammar issues.

Integration notes

- Lexer must heap-allocate strings (`safe_strdup`) and set semantic values; parser owns/freeing policy is in Annex C.

Annex C — AST Structures

Purpose

Compact overview of AST node shapes, ownership, construction in parser actions, and memory management.

Key concepts

- Core types: `Program` (root), `Stmt` (statement union), `Expr` (expression union), `CondExpr` (boolean conditions).
- Tagged enums: `StmtKind`, `ExprKind`, `CondKind` select the active union field for deterministic traversal.
- Constructors: Bison semantic actions call helper constructors (in `ast.cpp`) that `malloc()` nodes, populate fields, and return typed pointers via `%union`.

Ownership & memory

- Ownership is explicit: each node owns its heap-allocated fields (strings, child pointers).
- Freeing: call `ast_free_program(Program *p)` to recursively free nodes (post-order). Do not free nodes individually.

Typical node shapes

- `Program`: `Stmt **stmts, size_t count`.
- `Stmt`: `StmtKind kind + union (DeclStmt, PrintStmt, IfStmt, WhileStmt, FuncDef, FileOps, Collection ops, etc.)`.
- `Expr`: `ExprKind kind + union (literal, ident, binary, unary, call, index, convert, count/check)`.
- `CondExpr`: comparison/logic nodes with `Expr *` operands.

Bison integration

- `%union` contains `Expr *`, `Stmt *`, `Program *`, and list helpers; actions build nodes and pass pointers up the parse stack.

Annex D — Semantic Analysis

Purpose

Short reference for symbol table layout, type inference, semantic passes, and common diagnostics.

Phases

1. Collect function/routine signatures (first pass).
2. Validate statements, expressions, and calls using collected symbol/type info (second pass).

Symbol table

- `Symbol`: `name (char *)`, `TypeKind (TYPE_INT, TYPE_FLOAT, TYPE_STRING, TYPE_BOOL, TYPE_CHAR)`, `SymbolKind` declared size, scope level, declaration line.
- `SymbolTable` wraps a dynamic array or frames; lookups are linear/stack-based (educational scale).
- Scope: enter/leave tracked by `syntab_enter_scope()` / `syntab_leave_scope()` which push/pop frames and remove out-of-scope symbols.

Type inference & checking

- Recursive `infer_expr_type()` determines `TypeKind` for literals, identifiers (via symbol lookup), and expressions.
- Binary compatibility rules: `int+int -> int`, `int+float -> float` (promotion), `string+string -> string`; other mixes are errors.
- Function call validation checks existence, arity, and per-argument compatibility.

Diagnostics

Semantic errors are accumulated (non-fatal) to report multiple issues per run: undefined names, duplicate declarations, type mismatches, wrong arg counts.

Annex E — Code Generation & JIT

Purpose

Compact guide to lowering AST to LLVM IR, runtime helpers, JIT mapping, and IR output.

Overview

- Backend: LLVM C API for cross-version stability; use `LLVMContextRef`, `LLVMModuleRef`, `LLVMBuilderRef`.
- Emission: per-node emit functions generate `alloca/load/store/add/mul` or runtime calls for collections.
- Types: `TYPE_INT` -> `i64`, `TYPE_FLOAT` -> `double`, `TYPE_CHAR` -> `i8`, `TYPE_STRING` -> `i8*`, `TYPE_BOOL` -> `i1`.

Runtime helpers & JIT

- Complex ops (arrays, matrices, strings, file I/O) are implemented as native runtime functions and declared in the IR.
- At JIT time, use `LLVMAddGlobalMapping` (or equivalent) to map IR symbols to C/C++ function addresses.
- MCJIT compiles the module in-memory for the host (x86-64/ARM) and returns a function pointer to `main` for invocation.

Verification & output

- Call `LLVMVerifyModule` after emission. On success, optionally write `.ll` via `LLVMPrintModuleToFile()` for inspection.

Optimization & debug

- Apply LLVM passes (constant folding, dead-code elimination); use IR inspection when debugging invalid types or unmapped helpers.

Annex F — Tests & Limitations

Purpose

Condensed test plan, regression checklist, and known limitations for Lexico.

Test categories

- Lexical tests: unclosed strings, token length, INDENT/DEDENT mismatches (`test-lex-errors.lx`).
- Syntax tests: missing terminators, invalid operators unbalanced blocks (`test-syntax-errors.lx`).
- Semantic tests: undefined names, type mismatches, duplicate declarations (`test-semantic-errors.lx`).
- Valid programs: `emit-assign.lx`, `apps/passwordManager` for integration.

How to run

```
cmake -S . -B build-cmake
cmake --build build-cmake -j
./build/lexico.exe <test-file>.lx
```

Regression checklist

- Multi-word tokens recognized
- INDENT/DEDENT correct for nested blocks
- Function params and call arity checks
- Type promotion rules honored
- Runtime helper mapping present for JIT

Known limitations

- Scope: current implementation is largely flat/global; limited block scoping.
- Memory: manual `malloc/free` requires discipline; possible future migration to smart pointers.
- AI diagnostics: requires network and API key; autofix runs one retry only (safety).